
Quantum Principles, Structures and Computing

v1.0.0

Dr. Sreekar Guddeti

May 03, 2026

CONTENTS

1	Light matter interaction	3
1.1	Three quantum processes	3
1.2	Atomic spectra	4
1.3	Bohr's quantization rule	6
1.4	Coherence	7
1.5	Polarization	9
1.6	States of polarization	9
1.7	Interference	9
2	Quantum principles	11
2.1	Hilbert Space	11
2.2	Wave ket postulate	12
2.3	Operator postulate	13
2.4	Time evolution postulate	13
2.5	Measurement postulate	14
2.6	Commutator algebra	14
2.7	Pauli spin matrices	15
3	Quantum structures	17
3.1	Time independent Schrodinger equation	17
3.2	Numerical solution to differential equation	20
3.3	Numerical solution to TISE	27
3.4	Gaussian wavepacket	35
3.5	Scattering states	43
3.6	Quantum structures that host bound particle	56
3.7	Quantum confinement	57
4	Quantum Computing	59
4.1	Classical computation	59
4.2	Quantum computation	60
4.3	Classical simulation of quantum computation	60
4.4	Quantum simulation of classical computation	61
5	Qiskit	63
5.1	Installation	63
5.2	Qiskit objects	63
5.3	QuantumCircuit - setup single qubit circuits	63
5.4	Statevector	67
5.5	Operator	70
5.6	Registers - Measurement in Quantum Computation	75

5.7	Visualization of Quantum States	79
5.8	Setup multi qubit circuits	85
6	Home works	93
6.1	Waveket representation of polarization states	93
6.2	Eigen values and eigen kets of $\tilde{S}_x$	94
6.3	Unitary transformation	95
6.4	Dirac notation of Hermitian operator	97
6.5	Hamilonian representations in matrix and Dirac notation	98
6.6	Differentiation and integration of functions	99
6.7	Traveling waves vs standing waves	99
6.8	Anti Hermitian operators	99
7	Assignments	101
7.1	Assignment on light-matter interaction	101
7.2	Assignment on physics of smartphone	101
7.3	Assignment on linear system formulation	102
7.4	Assignment on postulates of quantum mechanics	102
7.5	Assignment on Gaussian wavepacket in free space	102
7.6	Assignment on TISE of quantum structures	102
7.7	Assignment on single qubit circuits	107
7.8	Assignment on multiple qubit circuits	107

Course structure

- This course titled “Quantum Principles, Structures, and Computing” is the first of the four courses that constitute the Minor program on Quantum Technology.
- As the name suggests, there are three parts to the course
 - Quantum Principles
 - Quantum Structures
 - Quantum Computing
- The course is framed into two modules with each module covering two units. While the first module focuses on the **conceptual aspects**, the second module covers the **application aspects**. Both the modules are covered in eight chapters.
- The conceptual part is covered in the first two chapters
 - Nature of light-matter interaction
 - Principles of quantum mechanics
- The application part is covered in the next three chapters
 - Quantum Structures
 - Quantum Computing
 - Qiskit
- The last two chapters enhance student’s learning aspects
 - Homeworks
 - Assignments

Prerequisites

The prerequisites for this course are

- Familiarity with basic linear algebra
- Familiarity with differentiation and integration
- Familiarity with Python

LIGHT MATTER INTERACTION

This is the introductory unit of this course. Here we will study the basics of light and matter interaction.

- *Three quantum process*
- *Atomic spectra*
- *Bohr's quantization rule*
- Properties of electromagnetic wave
 - *Coherence*
 - *Polarization*
- *Interference*

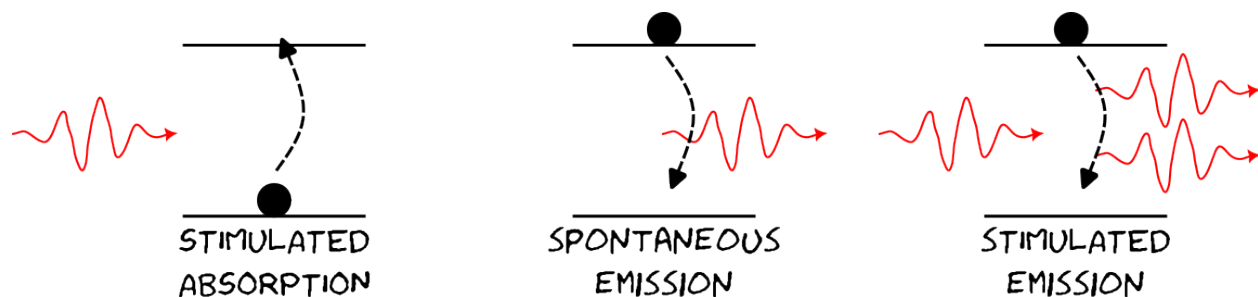
1.1 Three quantum processes

The problem of interaction of light and matter defines more or less the problem of physics because physics deals with the working of Nature; and Nature can be classified into light and matter.

After the advent of quantum mechanics, and the concept of quantization of energy levels, the interaction of light with matter can be broadly categorized into three quantum processes -

- Stimulated absorption
- Spontaneous emission
- Stimulated emission

Stimulated absorption forms the basis of absorption spectra, whereas spontaneous absorption forms the basis of emission spectra. The stimulated emission forms the basis of lasers and its discussion is out of the scope of this course.



1.2 Atomic spectra

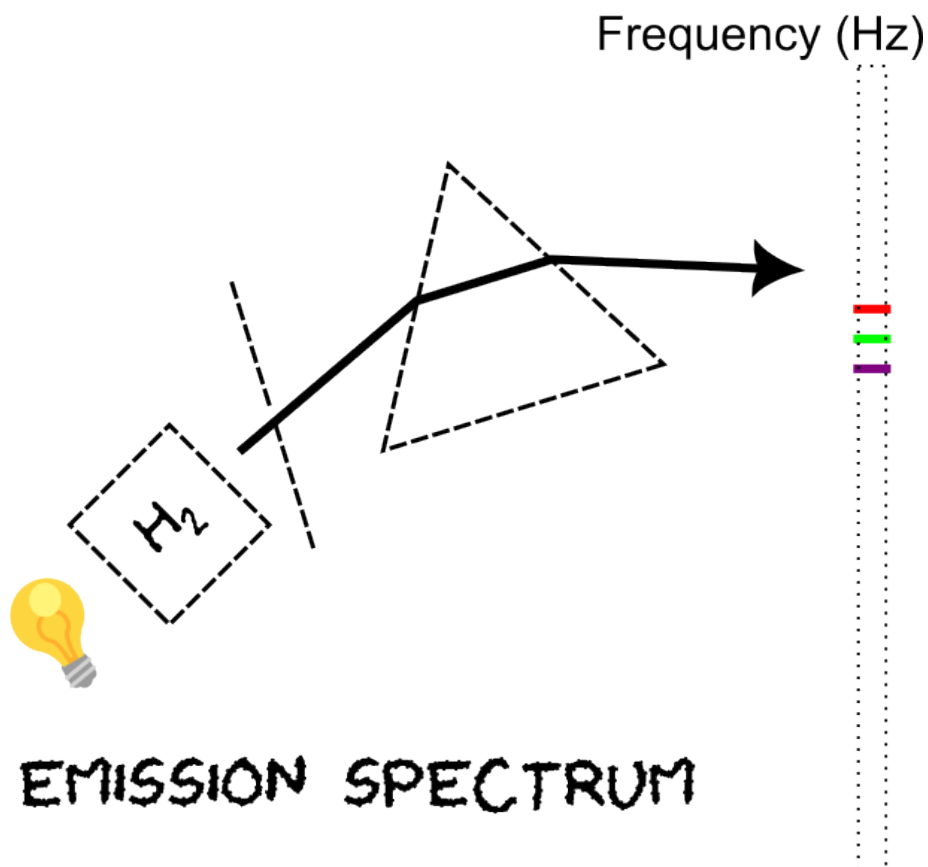
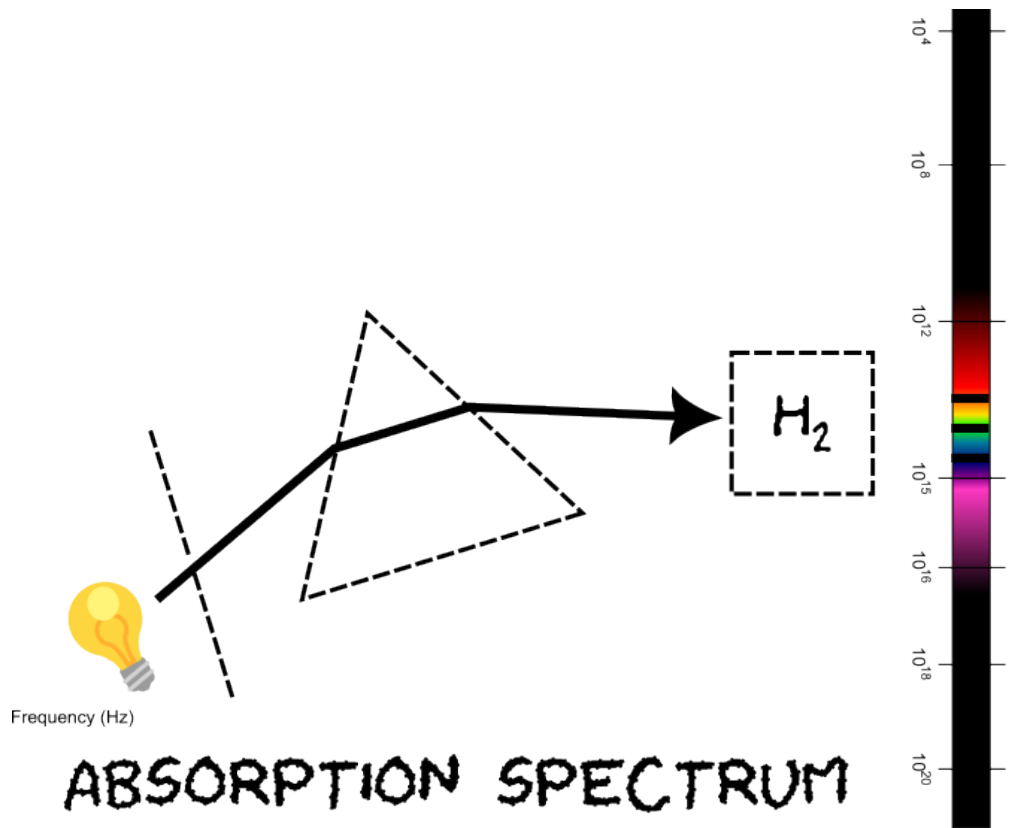
There are two kinds of atomic spectra

- absorption spectra
- emission spectrum

Early experiments on absorption spectra indicate that there are specific lines of absorption. These indicate that light is absorbed in specific energy by atoms.

These experiments were enhanced by applying external electric and magnetic fields leading to the Zeeman effect and Stark effect. Under the influence of the fields, the lines are further split into sub-lines.

These experiments needed theoretical basis which was provided by Bohr. Bohr proposed the quantum atom model to account for the lines of absorption of H atom.



1.3 Bohr's quantization rule

The quantization rule states that

the angular momentum of an electron can take integral multiples of reduced Planck's constant.

$$mvr = n\hbar$$

1.3.1 Energy levels of hydrogen atom

Bohr's model atom rests on the following two postulates.

- Electron revolves around the nucleus due to the Coulomb force of attraction.
- The angular momentum of electron is quantized and can take only integral multiples of $\hbar$.

Using Bohr's quantization rule and the Coulomb's law of interacting charges, let us derive the energy levels of the hydrogen atom.

If v is the velocity of the electron and r is the radius of orbit, then the first postulate translates to

$$\begin{aligned} F_{\text{Coulomb}} &= -\frac{mv^2}{r}, \\ -\frac{1}{4\pi\epsilon_0} \frac{e^2}{r^2} &= -\frac{m_0v^2}{r} \\ \Rightarrow v^2 &= \frac{1}{4\pi\epsilon_0} \frac{e^2}{m_0r} \end{aligned}$$

If $L = mvr$ is the angular momentum of electron, then the second postulate translates to

$$\begin{aligned} L &= n\hbar \\ \Rightarrow m_0vr &= \hbar \end{aligned}$$

where $n = 1$ is taken for the ground state.

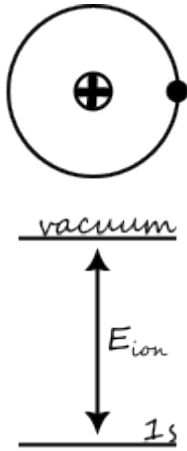
Bohr radius a_0 of H atom is the radius of electron's orbit in the ground state. From the above two equations, the Bohr radius is found to be

$$a_0 = \frac{4\pi\epsilon_0\hbar^2}{m_0e^2} = 0.529.$$

The total energy E_{1s} of the orbiting 1s electron is the sum of kinetic energy T and potential energy V so that

$$\begin{aligned} E_{1s} &= T + V \\ &= \frac{1}{2}m_0v^2 - \frac{1}{4\pi\epsilon_0} \frac{e^2}{r}, \\ &= -\frac{1}{2} \frac{m_0e^4}{\hbar^2 (4\pi\epsilon_0)^2} = -13.6 \end{aligned}$$

The Bohr model is applicable to **hydrogenic** or hydrogen-like atoms.



1.4 Coherence

Now that we understand how the light is absorbed or emitted in quantum of radiation, let us look at the light from a classical physics perspective

The model of light under classical perspective is that light is an electromagnetic wave with transverse electric and magnetic fields. the direction of propagation is perpendicular to both the fields.

This property is captured mathematically using vector algebra as

$$\vec{k} \parallel \vec{E} \times \vec{B}$$

So the electric and magnetic fields have two features

- wave nature of the fields
- transverse nature of the fields

Due the wave nature, the properties of light are governed the wave properties

- wavelength
- frequency
- amplitude
- phase
- principle of superposition

The simplest wave that can be imagined is a monochromatic plane wave. This assumes the electron has emitted from the excited to the ground state and we are studying the light at a distance very far from the electron. This can be mathematically represented as

$$\psi(y, t) = A \exp(\overline{ky - \omega t})$$

1.4.1 Incoherent light

However, if we consider bunch of electrons emitting the light, then not all the waves may be emitted in phase.

Since the waves follow the principle of superposition, the net amplitude of light is given by the sum of the individual amplitudes weighted by their **relative** phases. This results in incoherent light

Incoherent light is defined as the light with no definite phase relationship between any two adjacent points in space and time.

1.4.2 Coherent light

If the electrons emit light in phase, then the individual waves have a phase relationship. The net amplitude is non zero, and the resultant wave will have definite phase relationship between two adjacent points in space and time. This light is called a coherent light.

1.4.3 Spatial coherence

Both of the two cases are limiting cases. In practicality, the situation is somewhere in between. To measure the degree of coherence for such lights, we introduce the concepts of

- spatial coherence
- temporal coherence

1.4.4 Spatial coherence

The spatial coherence is a measure of how two points on wave that are spatially separated have a definite phase relationship.

What we mean by definite phase relationship is that the phase difference between the points should not change in other configuration axis, the time axis i.e.

If two points A and B spatially separated on the wave have phases ϕ_A and ϕ_B respectively, then the phase difference between them is

$$\Delta\phi = \phi_A - \phi_B$$

Then, we say the points A and B are spatially coherent when

$$\Delta\phi(t) = \text{constant}$$

Coherence length

It is understandable that closely spaced points are likely to be spatially coherent than far apart points.

Coherence length l_c is the maximum separation between points upto which the phase difference does not change by π in a time scale defined by the velocity of the wave

If the velocity is v , then the time scale is

$$\tau = \frac{v}{l_c}$$

#TODO is it group velocity or phase velocity?

1.4.5 Temporal coherence

Temporal coherence is a concept that is allied to the concept of spatial coherence. However, temporal coherence is the measure of how two points on wave that are **temporally** separated have a definite phase relationship.

Coherence time

Coherence time is a complementary concept to the coherence length.

1.5 Polarization

Due to the transverse nature of light, the light properties are governed by

- polarization

The polarization of light is a vector that is parallel to the electric field.

The polarization plane is the plane containing the electric field and is perpendicular to the direction of propagation.

1.6 States of polarization

Depending on the relation of polarization with time, the classical light is classified into

- linearly polarized light
- circularly polarized light
- elliptically polarized light

1.7 Interference

Interference is a result of principle of superposition. Since the waves add up according to the rules of vector algebra, we have two limiting cases

- constructive interference
- destructive interference

The other situations fall in between these cases

Let us study the concept of interference in the famous Young's double slit experiment

1.7.1 Young's double slit experiment

#TODO work on young's double slit experiment content

QUANTUM PRINCIPLES

Idea map

- *Hilbert space*
- Postulates of quantum mechanics
 - *Wave ket postulate*
 - *operator postulate*
 - *Time evolution postulate*
 - *Measurement postulate*
- *Commutator algebra*
- *Pauli's spin matrices*

2.1 Hilbert Space

Quantum states live in a mathematical structure called a Hilbert space.

We begin by recalling vectors in $\mathbb{R}^3$ and generalizing to complex vector spaces.

2.1.1 Inner Product

For vectors $|\psi\rangle$ and $|\varphi\rangle$:

$$\langle\varphi|\psi\rangle \in \mathbb{C}$$

Properties:

1. Conjugate symmetry
2. Linearity
3. Positive definiteness

$|\psi\rangle \rightarrow \text{ket}$

$\langle\psi| \rightarrow \text{bra}$

$\langle\varphi|\psi\rangle \rightarrow \text{inner product}$

```
import numpy as np

psi = np.array([1/np.sqrt(2), 1/np.sqrt(2)])
phi = np.array([1, 0])

inner = np.vdot(phi, psi)
inner
```

```
np.float64(0.7071067811865475)
```

1. Verify normalization of $|\psi\rangle = (1, i)/\sqrt{2}$
2. Check orthogonality of $(1,0)$ and $(0,1)$

2.2 Wave ket postulate

A quantum system is completely described by a normalized state vector $|\psi\rangle$.

2.2.1 Position representation

$$\psi(x) = \langle x | \psi \rangle$$

2.2.2 Normalization

$$\int |\psi(x)|^2 dx = 1$$

```
import numpy as np

x = np.linspace(-5, 5, 1000)
psi = np.exp(-x**2)
norm = np.trapezoid(np.abs(psi)**2, x)
norm
```

```
np.float64(1.2533141373155003)
```

2.2.3 Exercises

1. Normalize $\psi(x) = e^{-x^2}$
2. Interpret $|\psi(x)|^2$ physically

2.3 Operator postulate

Every observable corresponds to a linear Hermitian operator.

```
import numpy as np

sigma_z = np.array([[1,0],[0,-1]])
sigma_z
```

```
array([[ 1,  0],
       [ 0, -1]])
```

2.3.1 Eigenvalue equation

$$A|\psi\rangle = a|\psi\rangle$$

2.3.2 Exercises

1. Find eigenvalues of $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$
2. Show Hermitian condition $A = A^\dagger$

2.4 Time evolution postulate

$$i\hbar \frac{d}{dt}|\psi\rangle = H|\psi\rangle$$

2.4.1 Evolution operator

$$|\psi(t)\rangle = U(t)|\psi(0)\rangle$$

$$U(t) = e^{-iHt/\hbar}$$

```
# Two level system
import numpy as np
from scipy.linalg import expm

H = np.array([[1,0],[0,-1]])
t = 1
U = expm(-1j * H * t)
U
```

```
array([[0.54030231-0.84147098j, 0.          +0.j          ],
       [0.          +0.j          , 0.54030231+0.84147098j]])
```

2.4.2 Exercises

1. Show $U^\dagger U = I$
2. Compute evolution for σ_z Hamiltonian

2.5 Measurement postulate

Measurement outcomes = eigenvalues of operator

2.5.1 Probability

$$P(a) = |\langle a | \psi \rangle|^2$$

2.5.2 Collapse

After measurement $\rightarrow$ system collapses to eigenstate

```
import numpy as np
psi = np.array([1/np.sqrt(2), 1/np.sqrt(2)])
basis_0 = np.array([1, 0])

prob = abs(np.vdot(basis_0, psi))**2
prob
```

```
np.float64(0.4999999999999999)
```

2.5.3 Exercises

1. Compute probability for σ_z measurement
2. What happens after measurement?

2.6 Commutator algebra

The commutator between two operators $\tilde{A}$ and $\tilde{B}$ is defined as

$$[\tilde{A}, \tilde{B}] := \tilde{A}\tilde{B} - \tilde{B}\tilde{A}.$$

Commutator is a measure of how badly they fail to **commute**.

Two operators $\tilde{C}$ and $\tilde{D}$ are said to commute with each other when

$$\tilde{C}\tilde{D} = \tilde{D}\tilde{C},$$

so that

$$[\tilde{C}, \tilde{D}] = 0.$$

2.6.1 Canonical commutator relation

$$[x, p] = i\hbar$$

```
import numpy as np

A = np.array([[0, 1], [1, 0]])
B = np.array([[1, 0], [0, -1]])

comm = A @ B - B @ A
comm
```

```
array([[ 0, -2],
       [ 2,  0]])
```

2.6.2 Exercises

1. Compute $[\sigma_x, \sigma_y]$
2. Verify non-commutativity

2.7 Pauli spin matrices

$$\sigma_x = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

$$\sigma_y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$$

$$\sigma_z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

```
import numpy as np

sx = np.array([[0, 1], [1, 0]])
sy = np.array([[0, -1j], [1j, 0]])
sz = np.array([[1, 0], [0, -1]])
```

2.7.1 Angular momentum algebra

$$[\sigma_x, \sigma_y] = 2i\sigma_z$$

```
psi = np.array([1, 0]) # spin up
```

2.7.2 Exercises

1. Verify $\sigma_x^2 = I$
2. Compute eigenvalues of σ_y
3. Show commutation relations

QUANTUM STRUCTURES

Idea map

- *TISE*
- *Finite difference method*
- *Numerical solution to TISE*
- *Quantum structure leading to free particle*
- *Quantum structures leading to scattering particle*
- *Quantum structures leading to bound particle*
- *Quantum confinement*

3.1 Time independent Schrodinger equation

Let us derive the time independent Schrodinger equation for two cases -

- finite dimensional Hilbert basis
- infinite dimensional Hilbert basis

3.1.1 Finite dimensional Hilbert basis

The aim is to solve the Time Dependent Schrodinger equation (TDSE)

$$i\hbar \frac{d|\psi\rangle}{dt} = \tilde{H}|\psi\rangle$$

Stationary states

A stationary state $|\psi\rangle$ is an eigenstate of the Hamiltonian $\tilde{H}$ so that

$$\tilde{H}|\psi\rangle = E|\psi\rangle,$$

where E is the energy eigenvalue. This eigenvalue equation is called Time Independent Schrodinger Equation (TISE).

Let us consider the computational basis as the energy eigenbasis $\{|E_i\rangle\}$ where $i = \{1, 2, \dots, n\}$, where n is the dimension of the Hilbert space. Let us consider the initial state ($t = 0$) as a linear combination of the energy eigenkets as

$$|\psi(0)\rangle = \begin{pmatrix} c_1(0) \\ c_2(0) \\ \vdots \\ c_n(0) \end{pmatrix}$$

Since the Hamiltonian is diagonal in its own eigen basis, it is given by

$$\tilde{H} = \begin{pmatrix} E_1 & 0 & \dots & 0 \\ 0 & E_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & E_n \end{pmatrix}$$

Let us consider the general state at t as

$$|\psi(t)\rangle = \begin{pmatrix} c_1(t) \\ c_2(t) \\ \vdots \\ c_n(t) \end{pmatrix}$$

Upon substitution of $|\psi(t)\rangle$ and $\tilde{H}$ in TDSE, we have

$$i\hbar \begin{pmatrix} \frac{dc_1(t)}{dt} \\ \frac{dc_2(t)}{dt} \\ \vdots \\ \frac{dc_n(t)}{dt} \end{pmatrix} = \begin{pmatrix} E_1 c_1(t) \\ E_2 c_2(t) \\ \vdots \\ E_n c_n(t) \end{pmatrix}$$

We have a set of n first order differential equations

$$\begin{aligned} i\hbar \frac{dc_1(t)}{dt} &= E_1 c_1(t) \\ i\hbar \frac{dc_2(t)}{dt} &= E_2 c_2(t) \\ &\vdots \\ i\hbar \frac{dc_n(t)}{dt} &= E_n c_n(t) \end{aligned}$$

This set is reduced to

$$\frac{dc_1(t)}{dt} = -\frac{E_1}{i\hbar} c_1(t) \\ \frac{dc_2(t)}{dt} = -\frac{E_2}{i\hbar} c_2(t) \\ \vdots \\ \frac{dc_n(t)}{dt} = -\frac{E_n}{i\hbar} c_n(t)$$

The solutions to these differential equations have exponential form with the initial values serving as pre-factors

$$c_1(t) = c_1(0) \exp\left(-\frac{E_1 t}{i\hbar}\right) \\ c_2(t) = c_2(0) \exp\left(-\frac{E_2 t}{i\hbar}\right) \\ \vdots \\ c_n(t) = c_n(0) \exp\left(-\frac{E_n t}{i\hbar}\right)$$

The left hand side can be grouped together to represent the state at t whereas the right hand side is the solution of the TDSE given by

$$|\psi(t)\rangle = \begin{pmatrix} c_1(t) \\ c_2(t) \\ \vdots \\ c_n(t) \end{pmatrix} = \begin{pmatrix} c_1(0) \exp\left(-\frac{E_1 t}{i\hbar}\right) \\ c_2(0) \exp\left(-\frac{E_2 t}{i\hbar}\right) \\ \vdots \\ c_n(0) \exp\left(-\frac{E_n t}{i\hbar}\right) \end{pmatrix}$$

This can be decomposed into a linear combination of energy eigenkets

$$|\psi(t)\rangle = c_1(0) \exp\left(-\frac{E_1 t}{i\hbar}\right) |E_1\rangle + c_2(0) \exp\left(-\frac{E_2 t}{i\hbar}\right) |E_2\rangle + \dots + c_n(0) \exp\left(-\frac{E_n t}{i\hbar}\right) |E_n\rangle \\ = \sum_i c_i(0) \exp\left(-\frac{E_i t}{i\hbar}\right) |E_i\rangle$$

Q.E.D.

3.1.2 TISE in the coordinate basis

We have seen that the solution of the TDSE is intimately connected to the TISE

$$\tilde{H}|\psi\rangle = E|\psi\rangle,$$

where the Hamiltonian can be written in terms of the position $\tilde{X}$ and momentum $\tilde{P}$ operators as

$$\tilde{H}(\tilde{X}, \tilde{P}) = \frac{\tilde{P}^2}{2m} + V(\tilde{X}).$$

Here, $\tilde{T} = \frac{\tilde{P}^2}{2m}$ is the kinetic energy operator and $\tilde{V}(\tilde{X})$ is the potential energy operator.

Thus, we can express the Hamiltonian operator in the coordinate if we can express the position and momentum operators in the coordinate basis. The matrix elements of the operators are given by

$$\begin{aligned} \langle x | \tilde{X} | x' \rangle &= x \delta(x-x'), \quad \langle x | \tilde{P} | x' \rangle = \frac{\hbar}{i} \delta(x-x') \frac{d}{dx}. \end{aligned}$$

As a result, the matrix elements of operators $\tilde{T}$ and $\tilde{V}$ are given by

$$\begin{aligned} \langle x | \frac{\tilde{P}^2}{2m} | x' \rangle &= -\frac{\hbar^2}{2m} \delta(x-x') \frac{d^2}{dx^2} \\ \langle x | V(\tilde{X}) | x' \rangle &= V(x) \delta(x-x') \end{aligned}$$

As we are working in the coordinate basis, the completeness relation is given by

$$\tilde{I} = \int_{-\infty}^{\infty} |x'\rangle \langle x'| dx'$$

Let us plug the completeness relation inside the TISE, so that

$$\begin{aligned} \langle x | \tilde{H} | \psi \rangle &= E \langle x | \psi \rangle, \quad \langle x | \tilde{H} | \psi \rangle = \langle x | \tilde{T} + \tilde{V} | \psi \rangle \\ &= \langle x | \tilde{T} | \psi \rangle + \langle x | \tilde{V} | \psi \rangle \\ &= \int_{-\infty}^{\infty} \langle x | \tilde{T} | x' \rangle \langle x' | \psi \rangle dx' + \int_{-\infty}^{\infty} \langle x | \tilde{V} | x' \rangle \langle x' | \psi \rangle dx' \\ &= \int_{-\infty}^{\infty} \left(-\frac{\hbar^2}{2m} \delta(x-x') \frac{d^2}{dx'^2} + V(x) \delta(x-x') \right) \psi(x') dx' = E \psi(x). \end{aligned}$$

Take the inner product with $\langle x |$ on both sides of the equation so that

$$\begin{aligned} \int_{-\infty}^{\infty} \langle x | \tilde{H} | x' \rangle \langle x' | \psi \rangle dx' &= E \int_{-\infty}^{\infty} \langle x | x' \rangle \langle x' | \psi \rangle dx' \\ &= E \psi(x). \end{aligned}$$

Here, we have used the definition of wavefunction $\psi(x) := \langle x | \psi \rangle$.

Now,

$$\begin{aligned} \langle x | \tilde{H} | x' \rangle &= \langle x | \tilde{T} + \tilde{V} | x' \rangle \\ &= -\frac{\hbar^2}{2m} \delta(x-x') \frac{d^2}{dx'^2} + V(x) \delta(x-x'), \end{aligned}$$

Therefore, this is the matrix element of the Hamiltonian operator in the coordinate basis.

If we use the matrix element in the integral in Equation. (A), then

$$\int_{-\infty}^{\infty} \left(-\frac{\hbar^2}{2m} \frac{d^2}{dx'^2} + V(x) \right) \delta(x-x') \psi(x') dx' = E \psi(x).$$

Here, we have again used the definition of wavefunction $\psi(x') := \langle x' | \psi \rangle$.

Using the definition of delta function

$$\int_{-\infty}^{\infty} f(x') \delta(x-x') dx' = f(x),$$

we have

$$\left(-\frac{\hbar^2}{2m} \frac{d^2}{dx^2} + V(x)\right) \psi(x) = E \psi(x)$$

This is the TISE in the coordinate basis

$$Q.E.D.$$

3.2 Numerical solution to differential equation

Problem statement: Given a second order differential equation of dependent variable x and independent variable time t , and given the initial values (at time $t = 0$) for the position $x(0)$ and velocity $\dot{x}(0)$, the aim is to solve after time duration t , the position $x(t)$ and velocity $\dot{x}(t)$.

$$\ddot{x} = f(x, t)$$

The idea is to decompose the second order differential equation into two first order differential equations

Let us denote the first order derivative as velocity v and is given by

$$v := \dot{x} \quad \Rightarrow \quad \dot{x} = v$$

so that the second differential equation can be written in terms of x, t, v as

$$\ddot{x} = [\dot{v}] = \dot{v} = f(x, t)$$

The set of first order differential equations that are equivalent to the given second order differential equation is

$$\begin{aligned} \dot{x} &= v \\ \dot{v} &= f(x, t) \end{aligned}$$

3.2.1 Generalized position and velocity

If we represent the set of position and velocity as the **generalized position** of the particle, it can be represented as a ket that is a 2-column vector $|\psi\rangle = \begin{pmatrix} x \\ v \end{pmatrix}$

Here, ket resides in the two-dimensional space spanned by position and momentum. This space is called **phase space** of the particle.

Similarly, if we represent the set of rate of change of position and rate of change of velocity as the **generalized velocity** of the particle, it can be represented as a ket that is a 2-column vector

$$\frac{d|\psi\rangle}{dt} = \begin{pmatrix} \dot{x} \\ \dot{v} \end{pmatrix} = \begin{pmatrix} v \\ f(x, t) \end{pmatrix}$$

3.2.2 Finite difference formula for velocity

If we discretize the time axis with equally spaced time samples t_n with spacing $\Delta t = t_n - t_{n-1}$, then

$$\frac{d|\psi\rangle}{dt} \sim \frac{|\psi(t_n)\rangle - |\psi(t_{n-1})\rangle}{\Delta t}$$

If the samples of kets are denoted by $|\psi(t_n)\rangle = |\psi_n\rangle$, then

$$|\psi_n\rangle = \begin{pmatrix} x_n \\ v_n \end{pmatrix}$$

3.2.3 Recurrence relation

Due to the finite difference formula, the values of ket at timestamp n can be expressed in terms of values of ket at $n - 1$ and other independent variables.

$$\begin{aligned} |\psi_n\rangle &\approx |\psi_{n-1}\rangle + \Delta t |\dot{\psi}_n\rangle \\ \begin{pmatrix} x_n \\ v_n \end{pmatrix} &= \begin{pmatrix} x_{n-1} \\ v_{n-1} \end{pmatrix} + \Delta t \begin{pmatrix} \dot{x}_n \\ \dot{v}_n \end{pmatrix} \\ &= \begin{pmatrix} x_{n-1} + v_{n-1} \Delta t + f(x_n, t_n) (\Delta t)^2 \\ v_{n-1} + f(x_n, t_n) \Delta t \end{pmatrix} \end{aligned}$$

But let us express v_n on the RHS in terms of v_{n-1} so that

$$\begin{pmatrix} x_n \\ v_n \end{pmatrix} = \begin{pmatrix} x_{n-1} + v_{n-1} \Delta t + f(x_n, t_n) (\Delta t)^2 \\ v_{n-1} + f(x_n, t_n) \Delta t \end{pmatrix}$$

If the error propagation due to $(\Delta t)^2$ is neglected, then

$$\begin{pmatrix} x_n \\ v_n \end{pmatrix} = \begin{pmatrix} x_{n-1} + v_{n-1} \Delta t \\ v_{n-1} + f(x_n, t_n) \Delta t \end{pmatrix}$$

The RHS can be considered as a matrix transformation so that

$$\begin{pmatrix} x_n \\ v_n \end{pmatrix} = \begin{bmatrix} 1 & \Delta t \\ f(x_n, t_n) \Delta t & 1 \end{bmatrix} \begin{pmatrix} x_{n-1} \\ v_{n-1} \end{pmatrix}$$

3.2.4 Simple Harmonic Motion

$$\ddot{x} = -\omega^2 x$$

so that $f(x, t) = -\omega^2 x$. The evolver is

$$U = \begin{bmatrix} 1 & \Delta t \\ -\omega^2 \Delta t & 1 \end{bmatrix}$$

```
# Implementation of recurrence relation
import numpy as np
import matplotlib.pyplot as plt
```

```
dt = 1e-4
omega = 2
n = 10 # number of periods
ts = np.arange(0, 2.0*np.pi/omega*n, dt)
```

```
# initial ket (x0, v0) = (0, 1)
ket0 = np.array([0, 2])
```

```
ket0
```

```
array([0, 2])
```

```
def evolver(ket, step):
    x, v = ket
    stepper = np.array([[1, step], [-1.0*omega**2*step, 1]])
    evolved_ket = stepper.dot(ket)
    return evolved_ket
```

```
kets = np.zeros((len(ts),2))
```

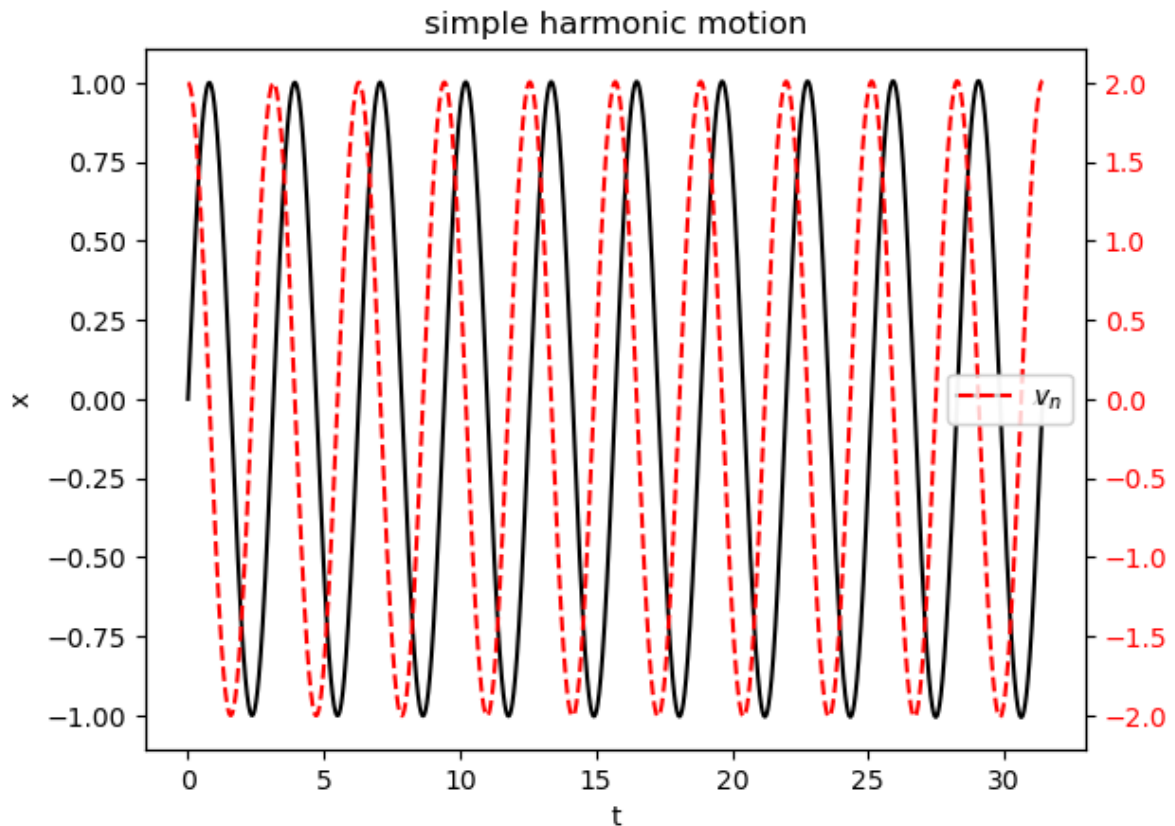
```
for i in range(len(ts)):
    if i==0:
        evolved_ket = ket0
        kets[i] = evolved_ket
    else:
        evolved_ket = evolver(kets[i-1], dt)
        kets[i] = evolved_ket
```

```
xs, vs = kets[:,0], kets[:,1]
```

```
fig, ax = plt.subplots()
ax.plot(ts, xs, 'k-', label=r'$x_n$')
ax2 = ax.twinx()
ax2.plot(ts, vs, 'r--', label=r'$v_n$')
ax.set_xlabel('t')
ax.set_ylabel('x')
ax.legend()

ax2.tick_params(labelcolor='r')
ax2.set_xlabel('v')
ax2.legend()
ax.set_title('simple harmonic motion')
```

```
Text(0.5, 1.0, 'simple harmonic motion')
```



3.2.5 Configuration space

The space spanned by the position coordinates of the system is called the configuration space. Here, the particle is in one-dimensional motion. Hence, the configuration space is one dimensional. However, not all of the axis is spanned. There are turning points due to the harmonic motion.

Since we started with initial velocity at origin, the particle reached maximum position x_{max} where the velocity is zero. Similarly, the other turning point is at x_{min} .

The trajectory of the particle can be represented as set of the $x(t), v(t)$ parametrized curves, where t is the parameter.

3.2.6 Phase space

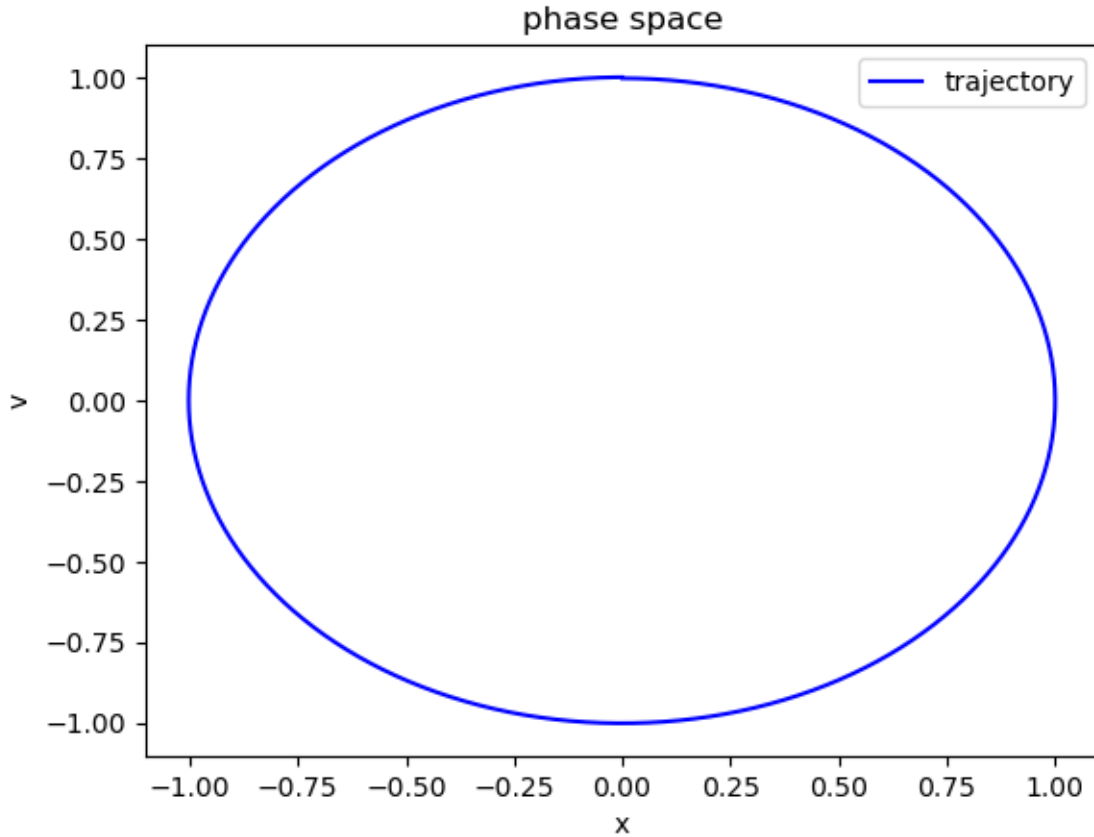
The space spanned by the position and velocity coordinates is called the phase space. The phase space of the particle in 1D simple harmonic motion will be two-dimensional.

The trajectory of the particle is represented as a single parametrized curve.

```
# Phase space visualization

fig, ax = plt.subplots()
ax.plot(xs, vs, 'b-', label='trajectory')
ax.set_xlabel('x')
ax.set_ylabel('v')
ax.set_title('phase space')
ax.legend()
```

```
<matplotlib.legend.Legend at 0x792af7b83ed0>
```



3.2.7 Acceleration due to gravity

Let us consider a two dimensional problem of a particle projected at an angle to the horizontal. The aim is to solve for the trajectory of the particle.

We have

$$\vec{a} = -g\hat{j},$$

where g is the acceleration due to gravity.

This is a vector equation that can be decomposed into two scalar equations.

$$\begin{aligned} \ddot{x} &= 0 \\ \ddot{y} &= -g \end{aligned}$$

If we denote the velocity components as

$$\vec{v} = v_x\hat{i} + v_y\hat{j}$$

Then the set of two second order differential equations can be decomposed as a set of four first order differential equations

$$\begin{aligned} \dot{x} &= v_x \\ \dot{y} &= v_y \\ \dot{v}_x &= 0 \\ \dot{v}_y &= -g \end{aligned}$$

Thus if the state of the system is represented by the 4-column ket

$$|\psi\rangle = \begin{pmatrix} x \\ y \\ v_x \\ v_y \end{pmatrix},$$

then the velocity of the state is given by

$$\frac{d|\psi\rangle}{dt} = \begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{v}_x \\ \dot{v}_y \end{pmatrix} = \begin{pmatrix} v_x \\ v_y \\ 0 \\ -g \end{pmatrix}$$

We can extend the finite difference method to four dimensional ket and solve for the trajectory.

The evolution is given by

$$|\psi_n\rangle = U|\psi_{n-1}\rangle + |\psi_0\rangle,$$

where the evolver is then given by

$$U = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

and

$$|\psi_0\rangle = \begin{pmatrix} 0 \\ 0 \\ 0 \\ -g\Delta t \end{pmatrix}$$

```
dt = 1e-3
g = 9.8 # acceleration due to gravity
alpha = np.pi/3.0 # np.pi/2.0 projectile angle
v0 = 1 # initial speed
T = 2.0*v0*np.sin(alpha)/g # time of flight
ts = np.arange(0, T, dt)
```

T

```
np.float64(0.1767398783233548)
```

```
# initial ket (x0, y0, vx0, vy0)
x0 = 0
y0 = 0
vx0 = v0*np.cos(alpha)
vy0 = v0*np.sin(alpha)
ket0 = np.array([x0, y0, vx0, vy0])
```

```
def evolver_projectile(ket, step):
    x, y, vx, vy = ket
    stepper = np.array([[1, 0, step, 0], [0, 1, 0, step], [0, 0, 1, 0], [0, 0, 0, 1]])
    offset = np.array([0, 0, 0, -1.0*g*step])
    evolved_ket = stepper.dot(ket) + offset
    return evolved_ket
```

```
kets = np.zeros((len(ts), 4))
```

```

for i in range(len(ts)):
    if i==0:
        evolved_ket = ket0
        kets[i] = evolved_ket
    else:
        evolved_ket = evolver_projectile(kets[i-1], dt)
        kets[i] = evolved_ket

```

```

xs, ys, vxs, vys = kets[:,0], kets[:,1], kets[:,2], kets[:,3]

```

```

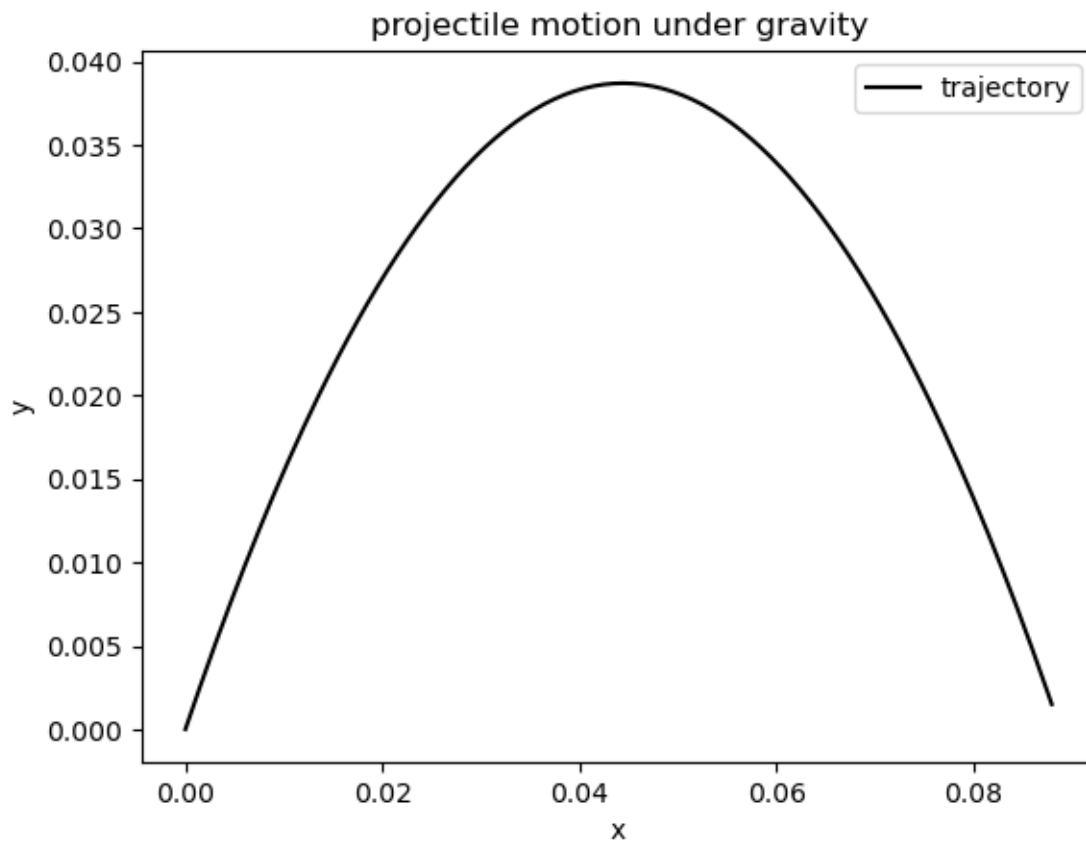
fig, ax = plt.subplots()
ax.plot(xs, ys, 'k-', label='trajectory')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.legend()
ax.set_title('projectile motion under gravity')

```

```

Text(0.5, 1.0, 'projectile motion under gravity')

```



3.3 Numerical solution to TISE

We have appreciated that the approach to solve for the Time Dependent Schrodinger Equation (TDSE) is through the solution of the Time Independent Schrodinger Equation (TISE)

We have also looked that the solution to second order differential equations numerically is to

1. find the appropriate set of first order differential equation for the state.
2. find the recurrence relation for the state.

Let us apply the numerical method to the TISE.

3.3.1 Step potential

Let us solve for the solution to the particle in a step potential.

$$V(x) = \begin{cases} 0 & x < 0 \\ 1 & x \geq 0 \end{cases}$$

The TISE equation is given by

$$\left(-\frac{\hbar^2}{2m} \frac{d^2}{dx^2} + V(x) \right) \psi(x) = E\psi(x)$$

Let us define the wave-vector $k(x)$ as the measure of the kinetic energy $T(x) = E - V(x)$ content of the particle at position x

$$k^2(x) = \frac{2mT(x)}{\hbar^2} = \frac{2m(E - V(x))}{\hbar^2}$$

Thus, the TISE can be written in terms of the wavevector $k(x)$ as

$$\frac{d^2 \psi(x)}{dx^2} + k^2(x) \psi(x) = 0$$

We have done a similar differential equation, the simple harmonic motion, where the parameter ω was a constant of motion. Here, the parameter $k(x)$ depends on the position of the particle.

So, let us apply the analysis of finite difference method to the TISE also and define the state of the particle by the 2-component ket $|\Psi\rangle$ as

$$|\Psi\rangle = \begin{pmatrix} \psi \\ v_\psi \end{pmatrix},$$

where v_ψ is the velocity of particle in space

$$v_\psi = \frac{d\psi(x)}{dx}$$

Since we are dealing with time axis evolution, let us call the matrix that gives the recurrence relation as propagator $\tilde{P}$, so that

$$|\Psi_n\rangle = \tilde{P}|\Psi_{n-1}\rangle,$$

where the propagator is given by

$$\tilde{P} = \begin{bmatrix} 1 & \Delta x \\ -k_{n-1}^2 \Delta x & 1 \end{bmatrix}$$

In terms of the eigen energy, the propagator is given by

$$\tilde{P} = \begin{bmatrix} 1 & \Delta x \\ -\frac{2m(E - V_{n-1})}{\hbar^2} \Delta x & 1 \end{bmatrix}$$

3.3.2 Initial state

Let the wavefunction at time $x = 0$ be $\psi(0)$. What is the velocity $v_\psi(0)$?

Or in other words, if the eigen energy is E , in the finite difference method, if the initial velocity is $(v_\psi)_0$, then let us make the physical assumption that far from the step, the kinetic energy does not change, therefore, after the propagates the state, the velocity of ψ is unchanged.

$$\begin{pmatrix} \psi_1 \\ (v_\psi)_1 \end{pmatrix} = \begin{bmatrix} 1 & \Delta x \\ -\frac{2m(E-V_{n-1})}{\hbar^2} \Delta x & 1 \end{bmatrix} \begin{pmatrix} \psi_0 \\ (v_\psi)_0 \end{pmatrix}$$

Here, we assume

$$(v_\psi)_1 = (v_\psi)_0,$$

so that

```
import numpy as np
import matplotlib.pyplot as plt
```

```
m = 1
hbar = 1
```

```
dx = 1e-4
xmin, xmax = -10.0, 10.0
V0 = 1
xs = np.arange(xmin, xmax+dx, dx)
```

```
def zero_potential(x):
    return 0.0
```

```
def step_potential(x):
    if x < 0.0:
        return 0.0
    else:
        return V0
```

```
# initial ket (psi0, vpsi0) = (0, 1)
ket0 = np.array([0, 1])
```

```
def propagator(ket, step, x, E, potential):
    psi, vpsi = ket
    #ksq = 2.0*(E-zero_potential(x))
    #ksq = 2.0*(E-step_potential(x))
    ksq = 2.0*(E-potential(x))
    stepper = np.array([[1, step], [-1.0*ksq*step, 1]])
    evolved_ket = stepper.dot(ket)
    return evolved_ket
```

```
def get_state(E, potential):
    """Return the wavefunction for a given energy"""
    kets = np.zeros((len(xs), 2))
    for i in range(len(xs)):
        x = xs[i]
        if i==0:
```

(continues on next page)

(continued from previous page)

```

        evolved_ket = ket0
        kets[i] = evolved_ket
    else:
        evolved_ket = propagator(kets[i-1], dx, x, E, potential)
        kets[i] = evolved_ket
state = kets
return state

```

```
state = get_state(1.5, step_potential)
```

```
pots = np.array([step_potential(x) for x in xs])
```

```
psis, vpsis = state[:,0], state[:,1]
```

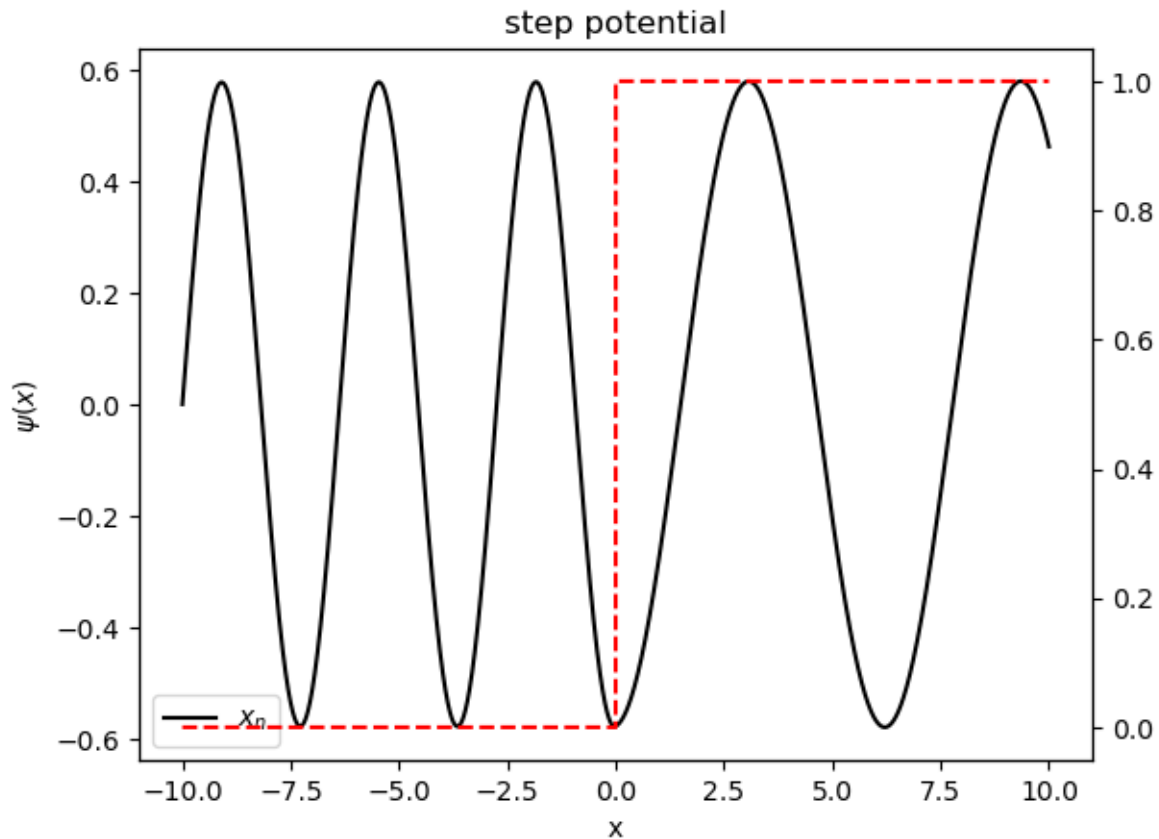
```

fig, ax = plt.subplots()
ax.plot(xs, psis, 'k-', label=r'$\psi_n$')
ax2 = ax.twinx()
ax2.plot(xs, pots, 'r--', label=r'$V(x)=0$')
ax.set_xlabel('x')
ax.set_ylabel(r'$\psi(x)$')
ax.legend()

#ax2.tick_params(labelcolor='r')
#ax2.set_xlabel('v')
#ax2.legend()
ax.set_title('step potential')

```

```
Text(0.5, 1.0, 'step potential')
```



3.3.3 Semi classical approximation

The analysis of step potential captures the different kinetic energies of the particle in the two regions. However, it fails to capture the amplitude variation that is a measure of the transmission and reflection of the wave at the edge.

If we assume that the potential is smoothly varying function of x instead of the step potential, then there is a possibility of capturing the reduction of the amplitude as the wave approaches the step.

The step function can be substituted by the Fermi-Dirac distribution at finite temperature

$$V(x) = V_0 \frac{1}{1 + \exp\left(\frac{-x}{T}\right)},$$

where T is a fictitious temperature with units of “position”. So in the neighbourhood of step with width of the order of T , the potential continuously varies from zero to V_0 .

```
# globals
```

```
T = 1
```

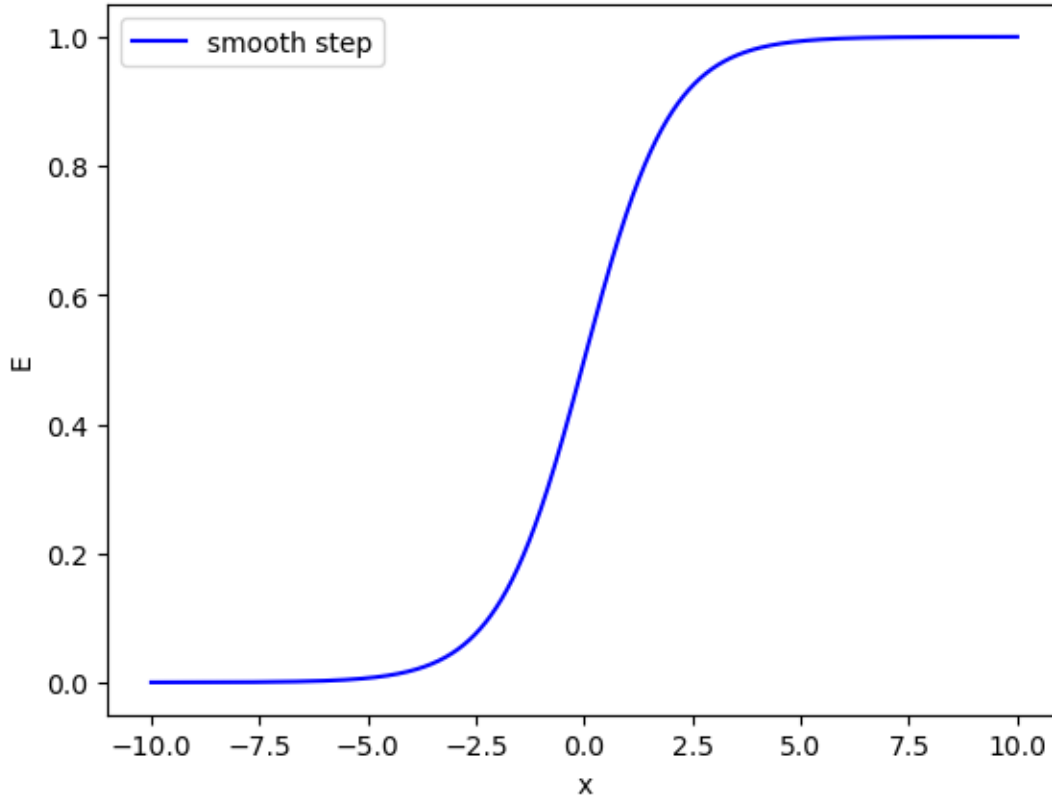
```
def smooth_step_potential(x):
    return V0/(1 + np.exp(-1.0*x/T))
```

```
pots = np.array([smooth_step_potential(x) for x in xs])
```

```
# Draw the smooth step.
```

```
fig, ax = plt.subplots()
ax.plot(xs, pots, 'b-', label='smooth step')
ax.set_xlabel('x')
ax.set_ylabel('E')
ax.legend()
```

```
<matplotlib.legend.Legend at 0x25efe3b8e20>
```



```
state = get_state(0.5, smooth_step_potential)
```

```
pots = np.array([smooth_step_potential(x) for x in xs])
```

```
psis, vpsis = state[:,0], state[:,1]
```

```
fig, ax = plt.subplots()
ax.plot(xs, psis, 'k-', label=r'$\psi_n$')
ax2 = ax.twinx()
ax2.plot(xs, pots, 'r--', label=r'$V(x) = V_0 \frac{1}{1 + \exp \left( \frac{-x}{T} \right)}$')
ax.set_xlabel('x')
ax.set_ylabel(r'$\psi(x)$')
ax.legend()

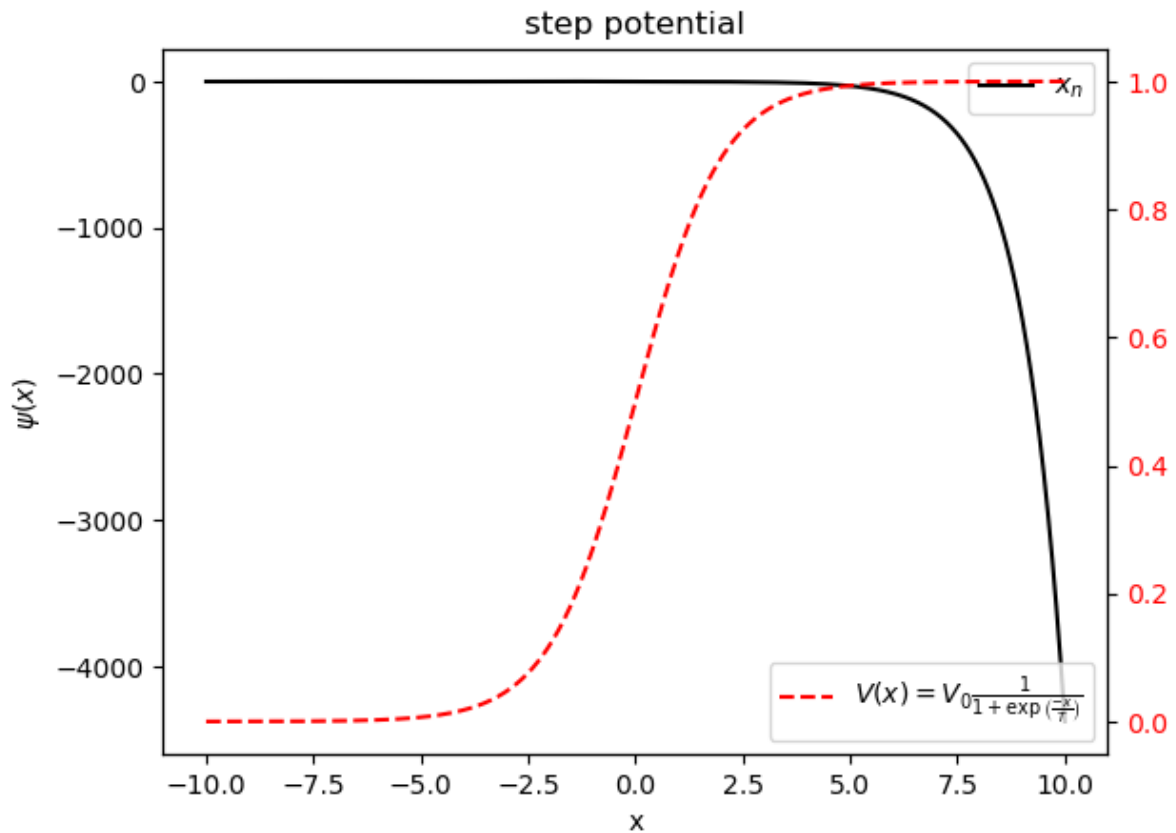
ax2.tick_params(labelcolor='r')
```

(continues on next page)

(continued from previous page)

```
ax2.set_xlabel('v')
ax2.legend(loc='lower right')
ax.set_title('step potential')
```

```
Text(0.5, 1.0, 'step potential')
```



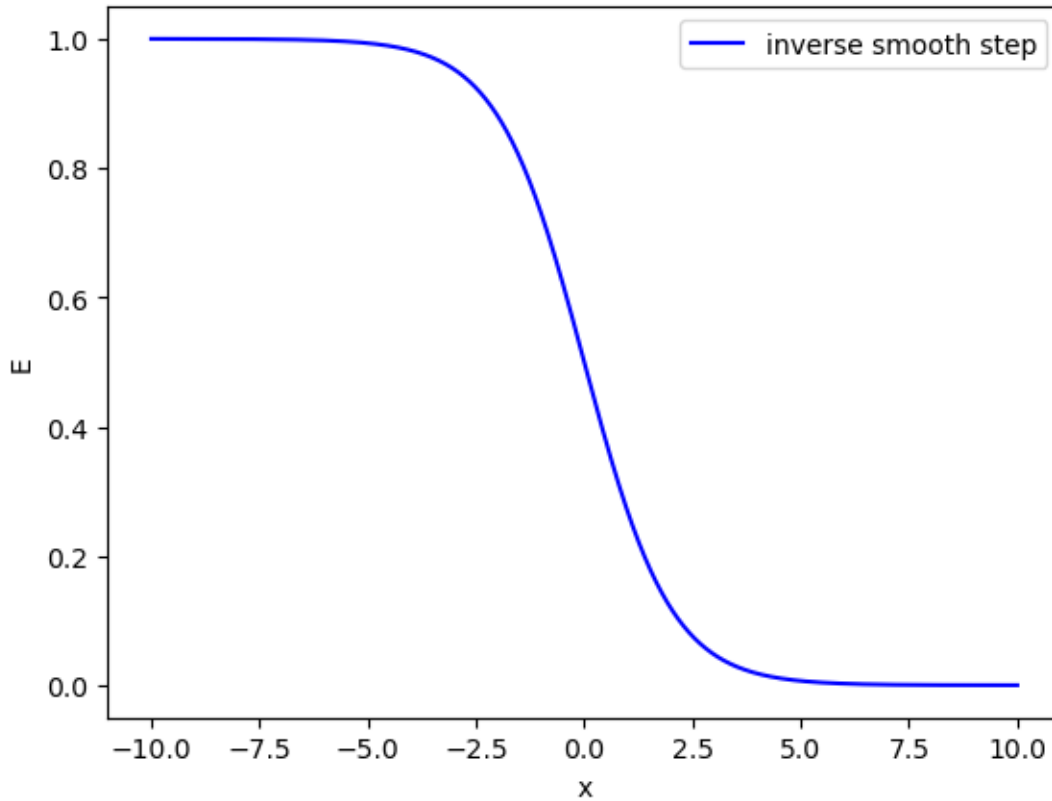
```
def inverse_smooth_step_potential(x):
    return smooth_step_potential(-1.0*x)
```

```
pots = np.array([inverse_smooth_step_potential(x) for x in xs])
```

```
# Draw the inverse smooth step.
```

```
fig, ax = plt.subplots()
ax.plot(xs, pots, 'b-', label='inverse smooth step')
ax.set_xlabel('x')
ax.set_ylabel('E')
ax.legend()
```

```
<matplotlib.legend.Legend at 0x25efcb3fbe0>
```



```
state = get_state(1.5, inverse_smooth_step_potential)
```

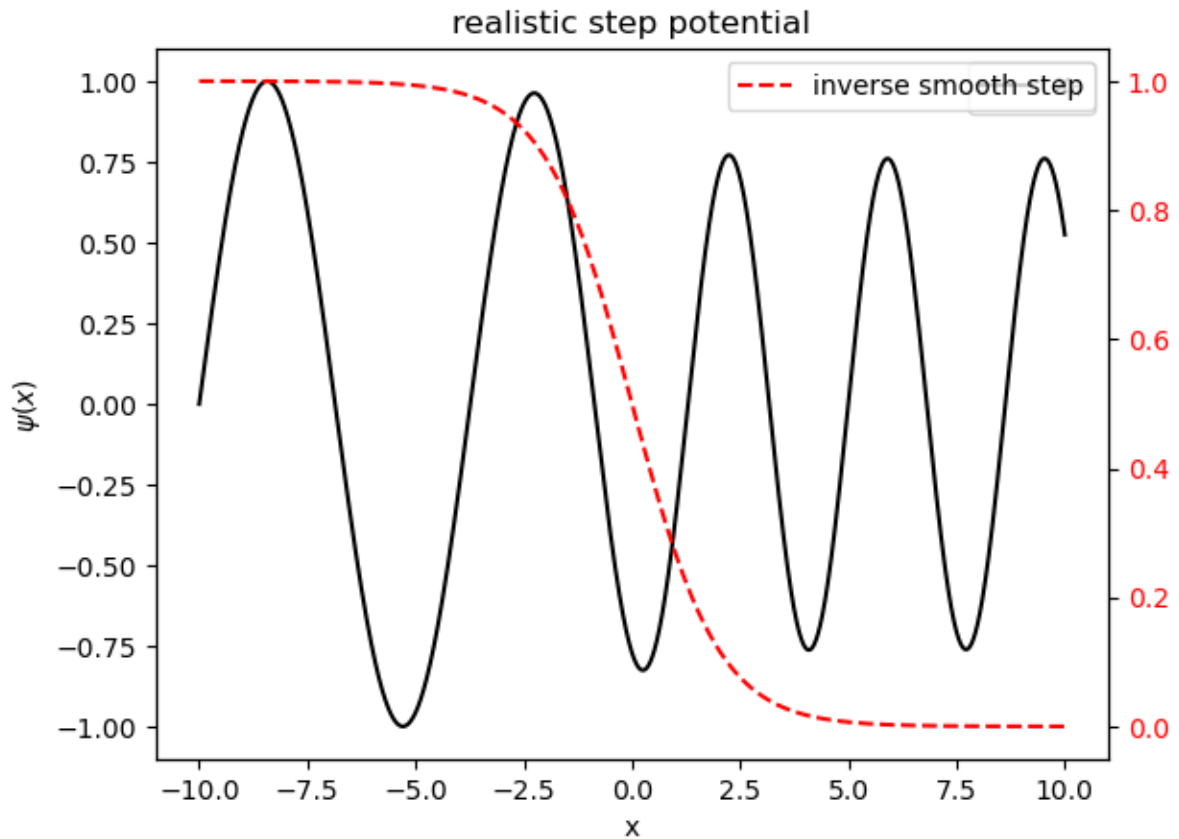
```
pots = np.array([inverse_smooth_step_potential(x) for x in xs])
```

```
psis, vpsis = state[:,0], state[:,1]
```

```
fig, ax = plt.subplots()
ax.plot(xs, psis, 'k-', label=r'$\psi_n$')
ax2 = ax.twinx()
ax2.plot(xs, pots, 'r--', label=r'inverse smooth step')
ax.set_xlabel('x')
ax.set_ylabel(r'$\psi(x)$')
ax.legend()

ax2.tick_params(labelcolor='r')
ax2.set_xlabel('V(x)')
ax2.legend()
ax.set_title('realistic step potential')
```

```
Text(0.5, 1.0, 'realistic step potential')
```



```
state = get_state(0.5, inverse_smooth_step_potential)
```

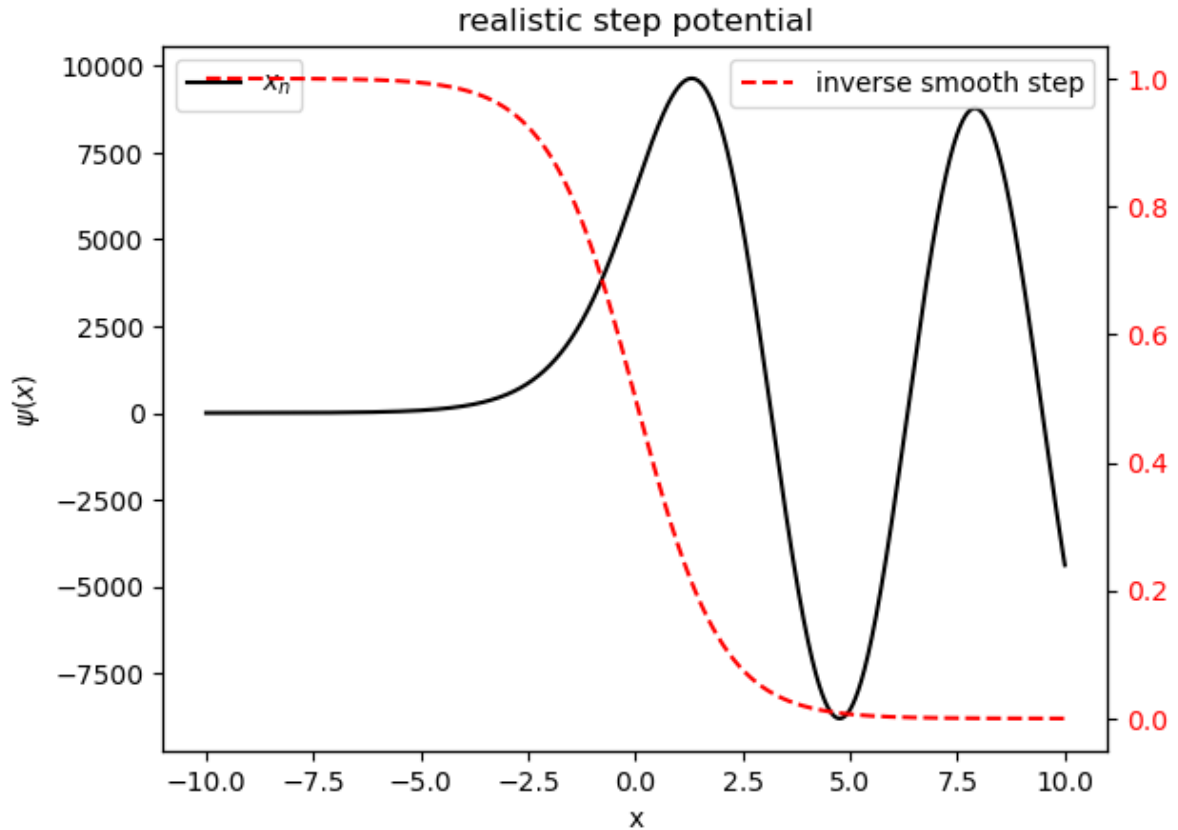
```
pots = np.array([inverse_smooth_step_potential(x) for x in xs])
```

```
psis, vpsis = state[:,0], state[:,1]
```

```
fig, ax = plt.subplots()
ax.plot(xs, psis, 'k-', label=r'$\psi_n$')
ax2 = ax.twinx()
ax2.plot(xs, pots, 'r--', label=r'inverse smooth step')
ax.set_xlabel('x')
ax.set_ylabel(r'$\psi(x)$')
ax.legend()

ax2.tick_params(labelcolor='r')
ax2.set_xlabel('V(x)')
ax2.legend()
ax.set_title('realistic step potential')
```

```
Text(0.5, 1.0, 'realistic step potential')
```



Let us create a widget based application that solves the TISE. The application needs to have the following widgets as a bare minimum

- Energy
- Initial state
- Choice of potential

The secondary widgets are

- step size

3.4 Gaussian wavepacket

The Gaussian wavepacket is given by

$$\psi(x, 0) = A \exp(-ax^2)$$

```
import numpy as np
from scipy.integrate import quad, quad_vec
from scipy.constants import hbar, m_e
```

```
A, A_vec = 1, np.asarray(1, dtype=np.complex128)
a, a_vec = 1, np.asarray(1, dtype=np.complex128)
hbar, m_e = np.asarray(hbar, dtype=np.complex128), np.asarray(m_e, dtype=np.complex128)
```

(continues on next page)

(continued from previous page)

```

↪complex128)
xmin, xmax = -10, 10
xmin_vec, xmax_vec = np.asarray(xmin, dtype=np.complex128), np.asarray(xmax, dtype=np.
↪complex128)
kmin, kmax = -10, 10
kmin_vec, kmax_vec = np.asarray(kmin, dtype=np.complex128), np.asarray(kmax, dtype=np.
↪complex128)

```

```
a_vec
```

```
array(1.+0.j)
```

```
a_vec.conjugate()
```

```
np.complex128(1-0j)
```

```

def gaussian(x):
    return A*np.exp(-1*a*x**2)

```

```

def gaussian_vec(x):
    x = np.asarray(x, dtype=np.complex128)
    value = A*np.exp(-1*a*x**2)
    return value.astype(np.complex128)

```

```

def integrand_of_norm(x):
    return gaussian(x)**2

```

```

def integrand_of_norm_vec(x):
    x = np.asarray(x, dtype=np.complex128)
    value = gaussian_vec(x)*gaussian_vec(x).conjugate()
    return value.astype(np.complex128)

```

```

def get_norm():
    norm = quad(integrand_of_norm, -np.inf, np.inf)
    return norm[0]

```

```

def get_norm_vec():
    """quad_vec() takes care of vectorized integrals"""
    result = quad_vec(integrand_of_norm_vec, -np.inf, np.inf)
    return result[0]

```

```

# test get_norm()
norm = get_norm()

```

```
norm
```

```
1.2533141373155017
```

```
A /= np.sqrt(norm)
```

A

```
np.float64(0.8932438417380018)
```

```
# test get_norm_vec()
norm_vec = get_norm_vec()
```

```
norm_vec
```

```
np.complex128(0.9999999999999999+0j)
```

The wavepacket is composed of k-waves with the coefficient $\phi(k)$

$$\psi(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{+\infty} \phi(k) \exp[+jkx] dk$$

where the coefficient is given by

$$\phi(k) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{+\infty} \psi(x) \exp[-jkx] dx$$

```
def integrand_of_coefficient(x, k):
    return 1/np.sqrt(2.0*np.pi)*gaussian(x)*np.exp(-1j*k*x)
```

```
def integrand_of_coefficient_vec(x, k):
    x = np.asarray(x, dtype=np.complex128)
    k = np.asarray(k, dtype=np.complex128)
    prefactor = 1/np.sqrt(2.0*np.pi)
    prefactor = np.asarray(prefactor, dtype=np.complex128)
    value = prefactor*gaussian_vec(x)*np.exp(-1j*k*x)
    return value.astype(np.complex128)
```

```
def get_coefficient(k):
    coefficient = quad(integrand_of_coefficient, -np.inf, np.inf, args=(k,))
    return coefficient[0]
```

```
def get_coefficient_vec(k):
    k = np.asarray(k, dtype=np.complex128)
    result = quad_vec(integrand_of_coefficient_vec, -np.inf, np.inf, args=(k,))
    coefficient = result[0]
    return coefficient
```

```
coeff = get_coefficient(1)
```

```
coeff
```

```
0.4919051987112381
```

```
coeff_vec = get_coefficient_vec(1)
```

```
coeff_vec
```

```
np.complex128(0.49190519871123844-1.3877801042704258e-17j)
```

```
coeff_vec.real, coeff_vec.imag
```

```
(np.float64(0.49190519871123844), np.float64(-1.3877801042704258e-17))
```

```
ks = np.linspace(xmin, xmax)
coeffs = [get_coefficient(k) for k in ks]
coeff_vecs = [get_coefficient_vec(k) for k in ks]
```

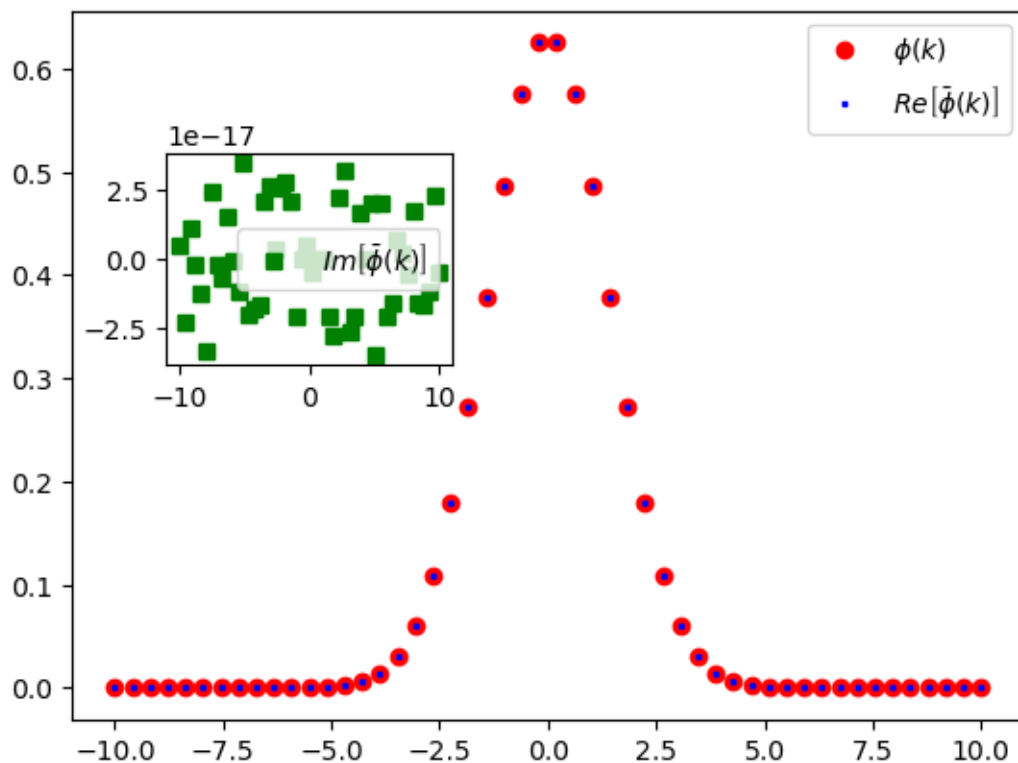
Let us plot the coefficient function and check if it is also a real valued function.

```
coeff_vec_reals = [each.real for each in coeff_vecs]
coeff_vec_imags = [each.imag for each in coeff_vecs]
```

```
import matplotlib as mpl
import matplotlib.pyplot as plt
```

```
fig, ax = plt.subplots()
ax.plot(ks, coeffs, 'ro', lw=3, label=r'$\phi(k)$')
ax.plot(ks, coeff_vec_reals, 'bs', lw=3, ms=2, label=r'$Re\left[\bar{\phi}(k)\right]$')
ax1 = ax.inset_axes([0.1, 0.5, 0.3, 0.3])
ax1.plot(ks, coeff_vec_imags, 'gs', label=r'$Im\left[\bar{\phi}(k)\right]$')
ax.legend()
ax1.legend()
```

```
<matplotlib.legend.Legend at 0x7d5879d82850>
```



3.4.1 Time evolution of wave packet

The wavepacket evolves in time. Each k-wave picks up a phase over time with the phase factor given by

$$\exp\left[-j\frac{E(k)t}{\hbar}\right],$$

where $E(k)$ is given by

$$E(k) = \frac{\hbar^2 k^2}{2m}$$

So the contribution to the wavepacket from k-wave changes from

$$\phi(k) \exp[+jkx]$$

to

$$\phi(k) \exp\left[-j\frac{E(k)t}{\hbar}\right] \exp[+jkx]$$

```
def energy(k):
    return hbar**2*k**2/(2*m_e)
```

```
def energy_vec(k):
    k = np.asarray(k, dtype=np.complex128)
    value = np.pow(hbar, 2+0j)*np.pow(k, 2+0j)/((2+0j)*m_e)
    return value.astype(np.complex128)
```

```
def integrand_of_wave(k, x, t):
    coefficient = get_coefficient(k)
    temporal_phase = np.exp(-1j*energy(k)*t/hbar)
    spatial_phase = np.exp(1j*k*x)
    return 1/np.sqrt(2.0*np.pi)*coefficient*temporal_phase*spatial_phase
```

```
def integrand_of_wave_vec(k, x, t):
    x = np.asarray(x, dtype=np.complex128)
    #k = np.asarray(k, dtype=np.complex128)
    t = np.asarray(t, dtype=np.complex128)
    prefactor = 1/np.sqrt(2.0*np.pi)
    prefactor = np.asarray(prefactor, dtype=np.complex128)
    coefficient_vec = get_coefficient_vec(k)
    temporal_phase_vec = np.exp(-1j*energy_vec(k)*t/hbar)
    spatial_phase_vec = np.exp(1j*k*x)
    value = prefactor*coefficient_vec*temporal_phase_vec*spatial_phase_vec
    return value.astype(np.complex128)
```

```
def wave(x, t):
    psi = quad(integrand_of_wave, kmin, kmax, args=(x, t))
    return psi[0]
```

```
def wave_vec(x, t):
    x = np.asarray(x, dtype=np.complex128)
    t = np.asarray(t, dtype=np.complex128)
    result = quad_vec(integrand_of_wave_vec, kmin, kmax, args=(x, t))
    value = result[0]
    return value.astype(np.complex128)
```

```
# test wave()
w, werror = quad(integrand_of_wave, kmin, kmax, args=(-2, 10))
```

```
/home/fubar/miniconda3/lib/python3.13/site-packages/scipy/integrate/_quadpack_py.
↳py:609: ComplexWarning: Casting complex values to real discards the imaginary
↳part
    return _quadpack._qagie(func, bound, infbounds, args, full_output,
/home/fubar/miniconda3/lib/python3.13/site-packages/scipy/integrate/_quadpack_py.
↳py:607: ComplexWarning: Casting complex values to real discards the imaginary
↳part
    return _quadpack._qagse(func, a, b, args, full_output, epsabs, epsrel, limit)
```

```
w
```

```
0.016360123344360353
```

```
# test wave_vec
w_vec, werror_vec = quad_vec(integrand_of_wave_vec, kmin, kmax, args=(-2, 10))
```

```
w_vec
```

```
np.complex128(0.016360123344357685+0.0001325802200246036j)
```

3.4.2 Spread of the wavepacket

Let us define the timescale of the problem

$$t_0 = \frac{m_e}{2\hbar a}$$

For $t \gg t_0$, the Full Width at Half Maximum (FWHM) of the wavepacket is given by

$$\text{FWHM}_{x(t)} \simeq 2\sqrt{\frac{\ln 2}{a}} \frac{t}{t_0}$$

```
t0 = m_e/(2.0*hbar*a)
```

```
t0
```

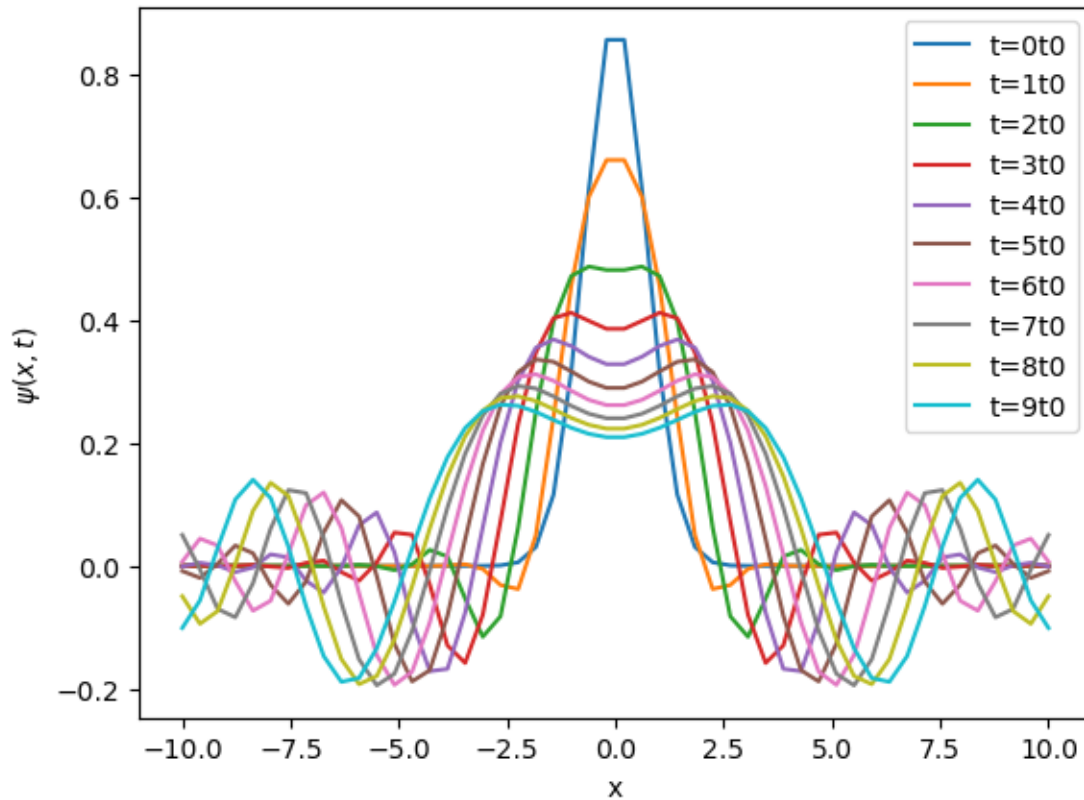
```
np.complex128(4318.996374392254+0j)
```

```
ts = np.linspace(0, 10*t0, 10)
xs = np.linspace(xmin, xmax, 50)
waves = np.zeros((len(ts), len(xs)))
wave_vecs = np.zeros((len(ts), len(xs)))
```

```
for i, t in enumerate(ts):
    for j, x in enumerate(xs):
        waves[i][j] = wave(x, t)
```

```
fig, ax = plt.subplots()
for i in range(len(waves)):
    ax.plot(xs, waves[i], label=f't={i}t0')
ax.set_xlabel('x')
ax.set_ylabel(r'$\psi(x,t)$')
ax.legend()
```

```
<matplotlib.legend.Legend at 0x7d5878748910>
```



```
for i, t in enumerate(ts):
    for j, x in enumerate(xs):
        wave_vecs[i][j] = wave_vec(x, t)
```

```
/tmp/ipykernel_410879/71966703.py:3: ComplexWarning: Casting complex values to
→real discards the imaginary part
    wave_vecs[i][j] = wave_vec(x, t)
```

```
wave_vecs[0][0]
```

```
np.float64(-5.273322985997755e-13)
```

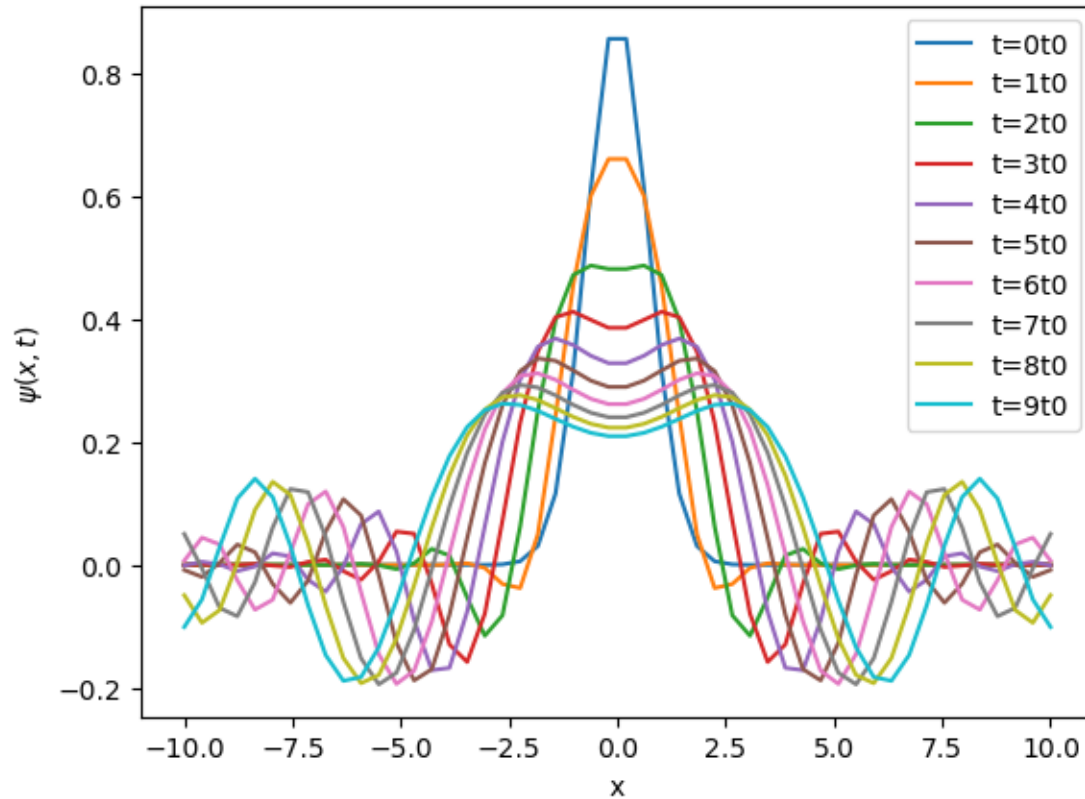
```
fig, ax = plt.subplots()
for i in range(len(wave_vecs)):
    ax.plot(xs, wave_vecs[i], label=f't={i}t0')
ax.set_xlabel('x')
ax.set_ylabel(r'$\psi(x,t)$')
```

(continues on next page)

(continued from previous page)

```
ax.legend()
```

```
<matplotlib.legend.Legend at 0x7d587886d590>
```



```
wave_vecs[0]
```

```
ts
```

```
xs
```

```
ks= np.linspace(-10, 10, 100)
w0s = [integrand_of_wave(k, -2, 0) for k in ks]
w, werror = quad(integrand_of_wave, -10, 10, args=(-2, 0))
```

```
fig, ax = plt.subplots()
ax.plot(ks, w0s, 'ro-', lw=3, label=r'$\phi(k) \exp \left[ -j \frac{E(k)t}{\hbar} \right] \exp \left[ +j k x \right]$')
ax.set_xlabel('k')
ax.legend()
```

3.5 Scattering states

A scattering state is a stationary state that has a free-particle like behaviour at infinity.

If the potential $V(x)$ tends to zero at infinity, then the energy eigenfunctions have scattering character, i.e. they have oscillatory behaviour extending upto infinity.

This is possible when the eigen energy of the scattering state is greater than zero.

The Time independent Schrodinger equation (TISE) is given by

$$\left(\frac{d^2}{dx^2} + \frac{2m}{\hbar^2} (E - V(x)) \right) \psi(x) = 0$$

Let the wavevector be defined as

$$k = \sqrt{\frac{2m}{\hbar^2} (E - V(x))},$$

so that TISE is reduced to

$$\left(\frac{d^2}{dx^2} + k^2 \right) \psi(x) = 0$$

3.5.1 Numerical solution to TISE

$$\frac{d^2\psi(x)}{dx^2} = -k^2\psi(x)$$

Using the definition of derivative,

$$\frac{d\psi(x)}{dx} = \lim_{h \rightarrow 0} \frac{\psi(x+h) - \psi(x)}{h},$$

so that the first derivative can be approximated by

$$\frac{d\psi(x)}{dx} \sim \frac{\psi(x+h) - \psi(x)}{h}, \quad h \ll 1$$

By the similar argument, it can be shown that the second order derivate can be approximated by

$$\frac{d^2\psi(x)}{dx^2} \sim \frac{\psi(x+h) - 2\psi(x) + \psi(x-h)}{h^2}, \quad h \ll 1$$

This is called the **finite difference formula** for the second order derivative.

3.5.2 Recurrence relation

Let us “discretize” the position axis into equally spaced points with the spacing between consecutive points given by h_0 .

Let us denote the value of the wave function at position x_n as

$$\psi_n := \psi(x_n)$$

Thus $\psi_0, \psi_1, \dots, \psi_n, \dots$ form a sequence. The aim is to find a recurrence relation between consecutive ψ_n 's. A recurrence relation gives the value of an element in a sequence in terms of the previous elements of the sequence.

3.5.3 Numerov's recurrence relation for TISE

The forward element $\psi(x + h_0)$ can be expressed in terms of the present element $\psi(x)$ using the Taylor's series expansion as

$$\psi(x_n + h_0) = \psi(x_n) + h_0\psi'(x_n) + \frac{h_0^2}{2}\psi''(x_n) + \frac{h_0^3}{6}\psi'''(x_n) + \frac{h_0^4}{24}\psi^{(4)}(x_n) + \frac{h_0^5}{120}\psi^{(5)}(x_n) + \mathcal{O}(h_0^6)$$

so

$$\psi_{n+1} = \psi_n + h_0\psi'_n + \frac{h_0^2}{2}\psi''_n + \frac{h_0^3}{6}\psi'''_n + \frac{h_0^4}{24}\psi^{(4)}_n + \frac{h_0^5}{120}\psi^{(5)}_n + \mathcal{O}(h_0^6)$$

Similarly

$$\psi_{n-1} = \psi_n - h_0\psi'_n + \frac{h_0^2}{2}\psi''_n - \frac{h_0^3}{6}\psi'''_n + \frac{h_0^4}{24}\psi^{(4)}_n - \frac{h_0^5}{120}\psi^{(5)}_n + \mathcal{O}(h_0^6)$$

Adding the above two equations gives

$$\psi_{n+1} + \psi_{n-1} = 2\psi_n + h_0^2\psi''_n + \frac{h_0^4}{12}\psi^{(4)}_n + \mathcal{O}(h_0^6)$$

The right hand side has two derivative terms. The second order derivative can be expressed in terms of TISE as

$$\psi''_n = -k_n^2\psi_n,$$

whereas the fourth order derivative can be expressed as

$$\psi^{(4)}_n = (\psi''_n)'' = -(k_n^2\psi_n)''$$

so that second order derivative can be **approximated** by the finite difference formula

$$(k_n^2\psi_n)'' \sim \frac{k_{n+1}^2\psi_{n+1} - 2k_n^2\psi_n + k_{n-1}^2\psi_{n-1}}{h_0^2}$$

Thus the recurrence relation is given by

$$\psi_{n+1} + \psi_{n-1} = 2\psi_n - h_0^2k_n^2\psi_n - \frac{h_0^4}{12} \frac{k_{n+1}^2\psi_{n+1} - 2k_n^2\psi_n + k_{n-1}^2\psi_{n-1}}{h_0^2}$$

Let us express ψ_{n+1} in terms of ψ_n and ψ_{n-1} so that

$$\psi_{n+1} = \frac{2\psi_n \left(1 - \frac{5h_0^2k_n^2}{12}\right) - \psi_{n-1} \left(1 + \frac{h_0^2k_{n-1}^2}{12}\right)}{1 + \frac{h_0^2k_{n+1}^2}{12}}$$

This is called Numerov's recurrence relation.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.constants import hbar, m_e
```

```
# global variables
xmin, xmax = -10.0, +10.0
hbar=1
m_e=1
```

```
# step potential
```

```
def step(x):
    if x < 0:
        return 0.0
    else:
        return 1.0
```

```
def inverse_step(x):
    return 1.0 * step(-1.0*x)
```

```
h_0 = 1e-2
epsilon = 1e-2
psi_0, psi_1 = 0, epsilon
xs = np.arange(xmin, xmax, h_0)
```

```
def numerov(E, potential):
    """The initial two elements in the sequence {psi_n} can be chosen as 0 and epsilon
    ↪"""
    psls = np.zeros(len(xs))
    # Get the ksquareds
    #ksquareds = np.array([2.0*m_e*(E - step(x))/hbar**2 for x in xs])
    ksquareds = np.array([2.0*m_e*(E - potential(x))/hbar**2 for x in xs])
    for i in range(len(psls)):
        if i == 0:
            psls[i] = psi_0
        elif i == 1:
            psls[i] = psi_1
        else:
            numer = 2.0*psls[i-1]*(1.0 - 5.0/12.0*h_0**2*ksquareds[i-1]) - psls[i-
            ↪2]*(1.0 + 1.0/12.0*h_0**2*ksquareds[i-2])
            denom = 1 + 1.0/12.0*h_0**2*ksquareds[i]
            psls[i] = numer/denom
    return psls
```

```
# Initial wave function
psi_0, psi_1 = 0, 1.0*epsilon
```

```
# Approximate wave function
E = 0.5
psls = np.zeros(len(xs))
```

```
psls = numerov(E, step)
pots = np.array([step(x) for x in xs])
```

```
psls
```

```
array([ 0.00000000e+00,  1.00000000e-02,  1.99990000e-02, ...,
        -1.47241875e+04, -1.48721680e+04, -1.50216358e+04], shape=(2000,))
```

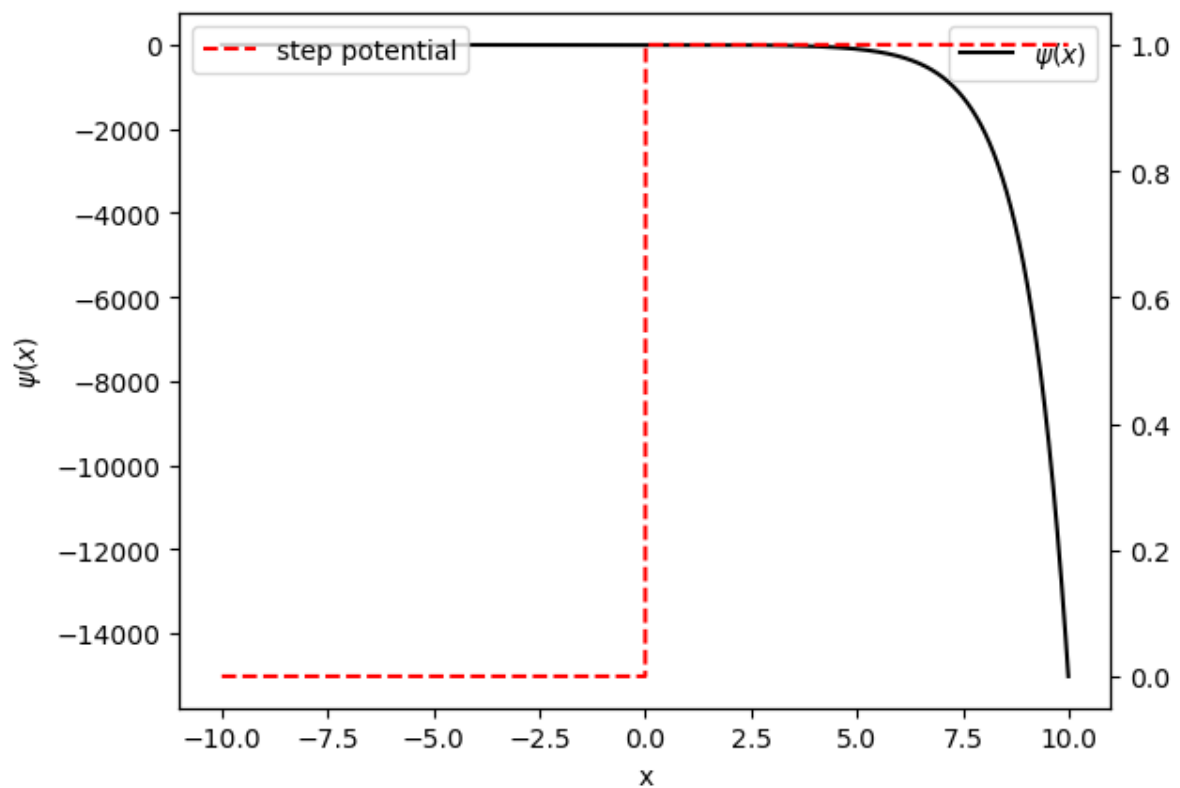
```
fig, ax = plt.subplots()
ax.plot(xs, psls, 'k-', label=r'$\psi(x)$')
ax2 = ax.twinx()
ax2.plot(xs, pots, 'r--', label='step potential')
```

(continues on next page)

(continued from previous page)

```
ax.set_xlabel('x')
ax.set_ylabel(r'$\psi(x)$')
ax.legend()
ax2.legend()
```

```
<matplotlib.legend.Legend at 0x186b593de50>
```



```
# Initial wave function
psi_0, psi_1 = 0, -1.0*epsilon
```

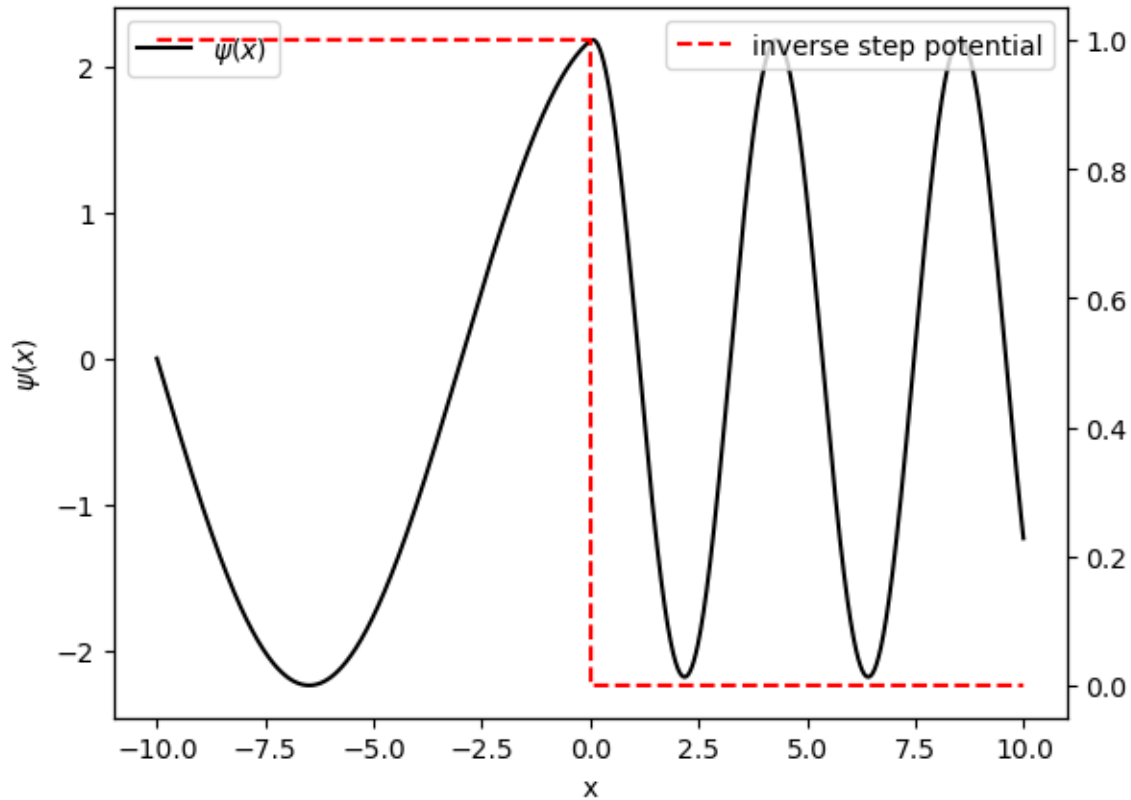
```
# Approximate wave function
E = 1.1
psis = np.zeros(len(xs))
```

```
psis = numerov(E, inverse_step)
pots = np.array([inverse_step(x) for x in xs])
```

```
fig, ax = plt.subplots()
ax.plot(xs, psis, 'k-', label=r'$\psi(x)$')
ax2 = ax.twinx()
ax2.plot(xs, pots, 'r--', label='inverse step potential')

ax.set_xlabel('x')
ax.set_ylabel(r'$\psi(x)$')
ax.legend()
ax2.legend()
```

```
<matplotlib.legend.Legend at 0x186b5d7ee90>
```



3.5.4 Smooth step potential

However, the step potential is not capturing the change in the amplitude of the waves in the two regions. This is because of the “abruptness” of the potential. In nature, nothing is abrupt. So let us substitute the step potential with a more realistic potential

$$V(x) = V_0 \frac{1}{1 + \exp(-\frac{x}{T})},$$

where T is a fictitious temperature with units in position.

```
# globals
T = 1
V0 = 1
```

```
def smooth_step(x):
    return V0 / (1 + np.exp(-1.0*x/T))
```

```
pots = np.array([smooth_step(x) for x in xs])
```

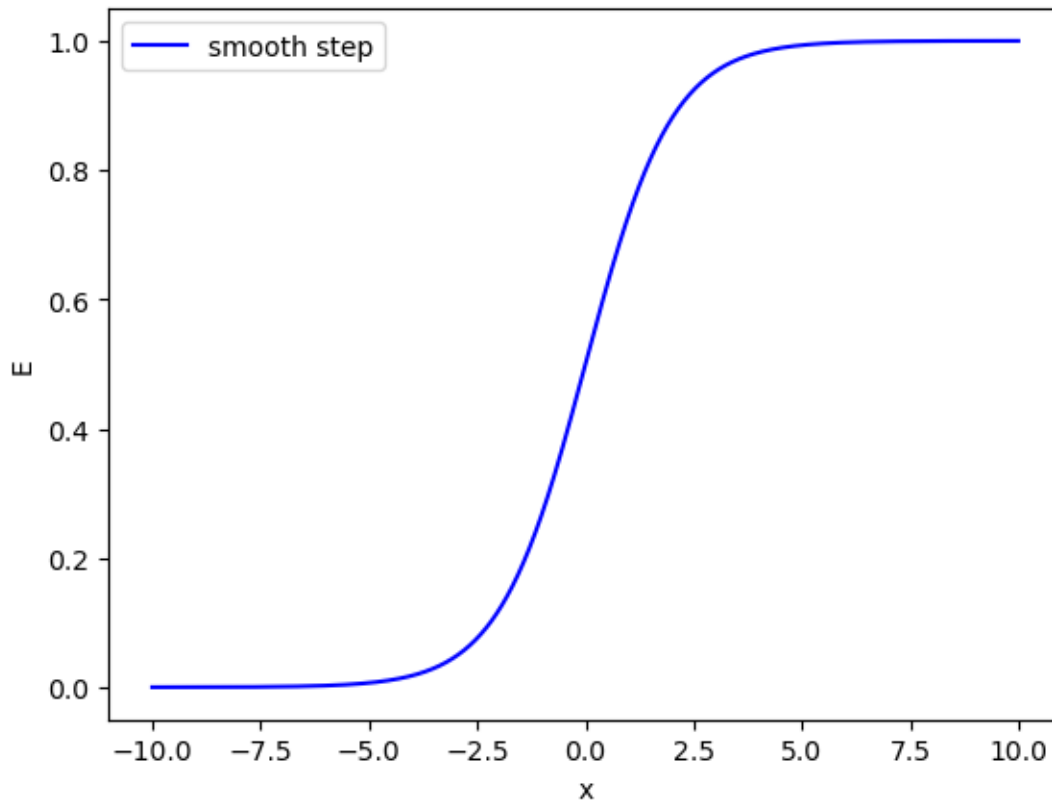
```
fig, ax = plt.subplots()
ax.plot(xs, pots, 'b-', label='smooth step')
ax.set_xlabel('x')
```

(continues on next page)

(continued from previous page)

```
ax.set_ylabel('E')
ax.legend()
```

```
<matplotlib.legend.Legend at 0x186b5c06850>
```



```
# Initial wave function
psi_0, psi_1 = 0, 1.0*epsilon
```

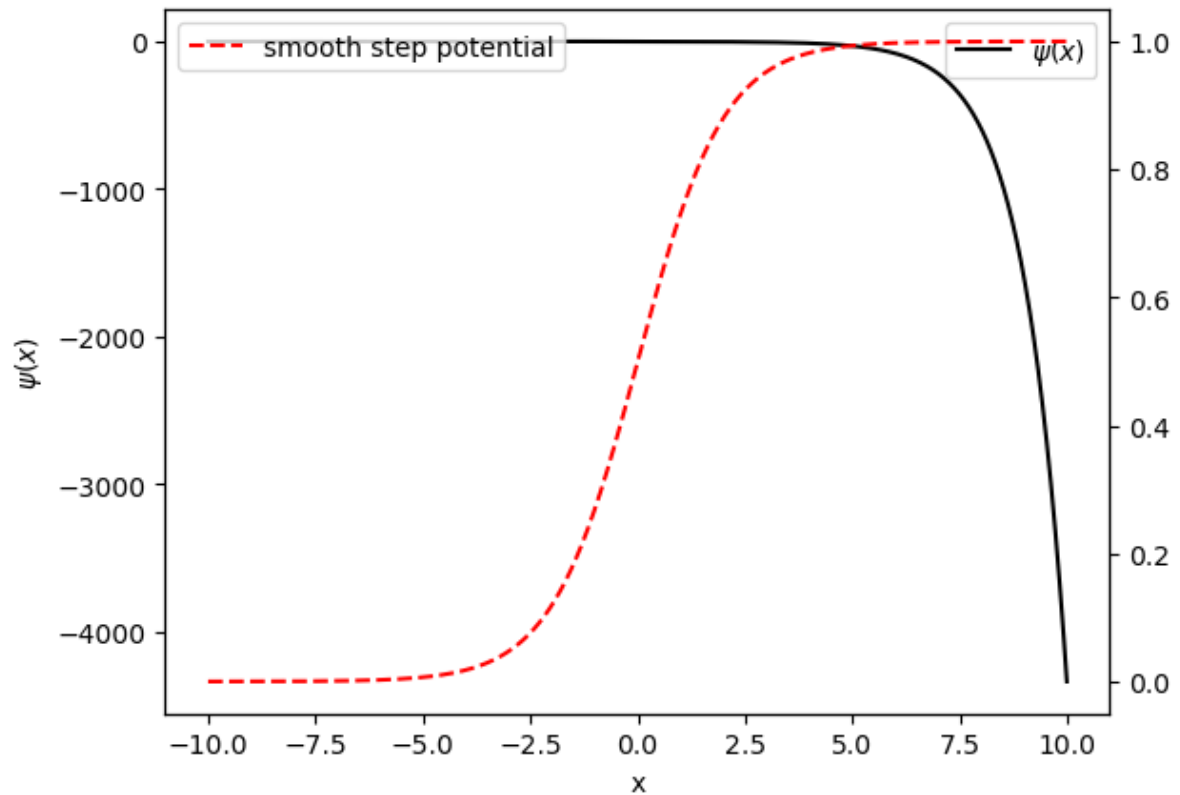
```
# Approximate wave function
E = 0.5
psis = np.zeros(len(xs))
```

```
psis = numerov(E, smooth_step)
pots = np.array([smooth_step(x) for x in xs])
```

```
fig, ax = plt.subplots()
ax.plot(xs, psis, 'k-', label=r'$\psi(x)$')
ax2 = ax.twinx()
ax2.plot(xs, pots, 'r--', label='smooth step potential')

ax.set_xlabel('x')
ax.set_ylabel(r'$\psi(x)$')
ax.legend()
ax2.legend()
```

```
<matplotlib.legend.Legend at 0x186b5a66710>
```

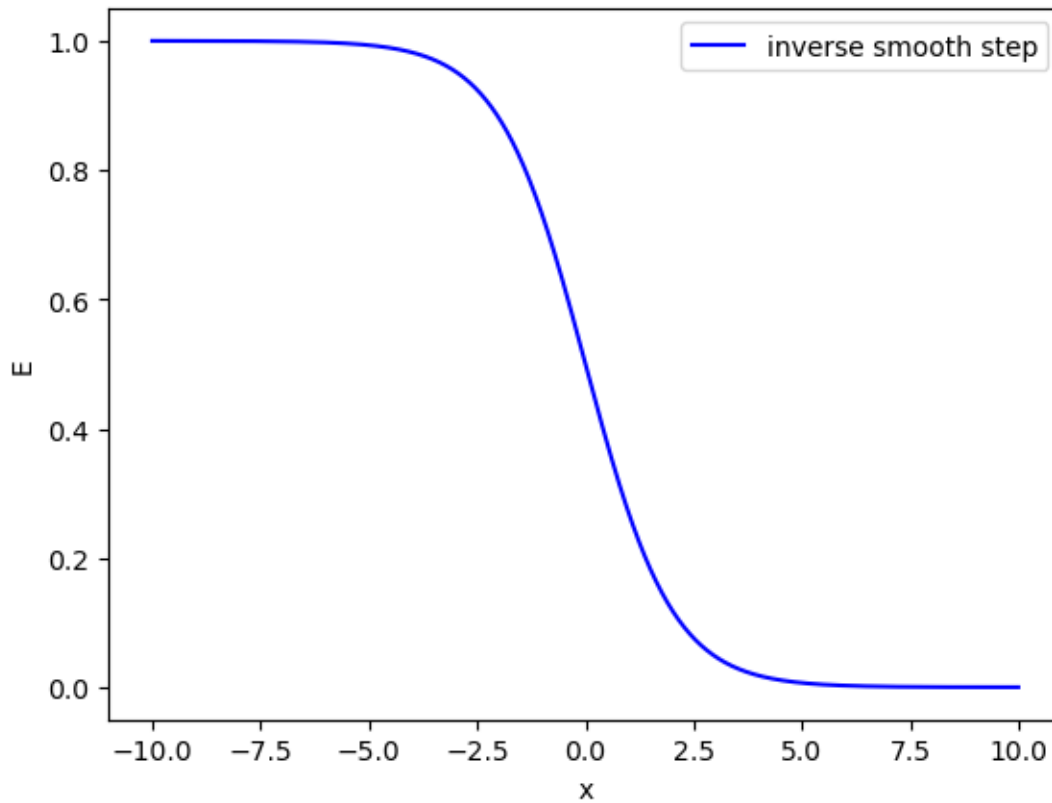


```
def inverse_smooth_step(x):
    return smooth_step(-1.0*x)
```

```
pots = np.array([inverse_smooth_step(x) for x in xs])
```

```
fig, ax = plt.subplots()
ax.plot(xs, pots, 'b-', label='inverse smooth step')
ax.set_xlabel('x')
ax.set_ylabel('E')
ax.legend()
```

```
<matplotlib.legend.Legend at 0x186b610de50>
```



```
# Initial wave function
psi_0, psi_1 = 0, 1.0*epsilon
```

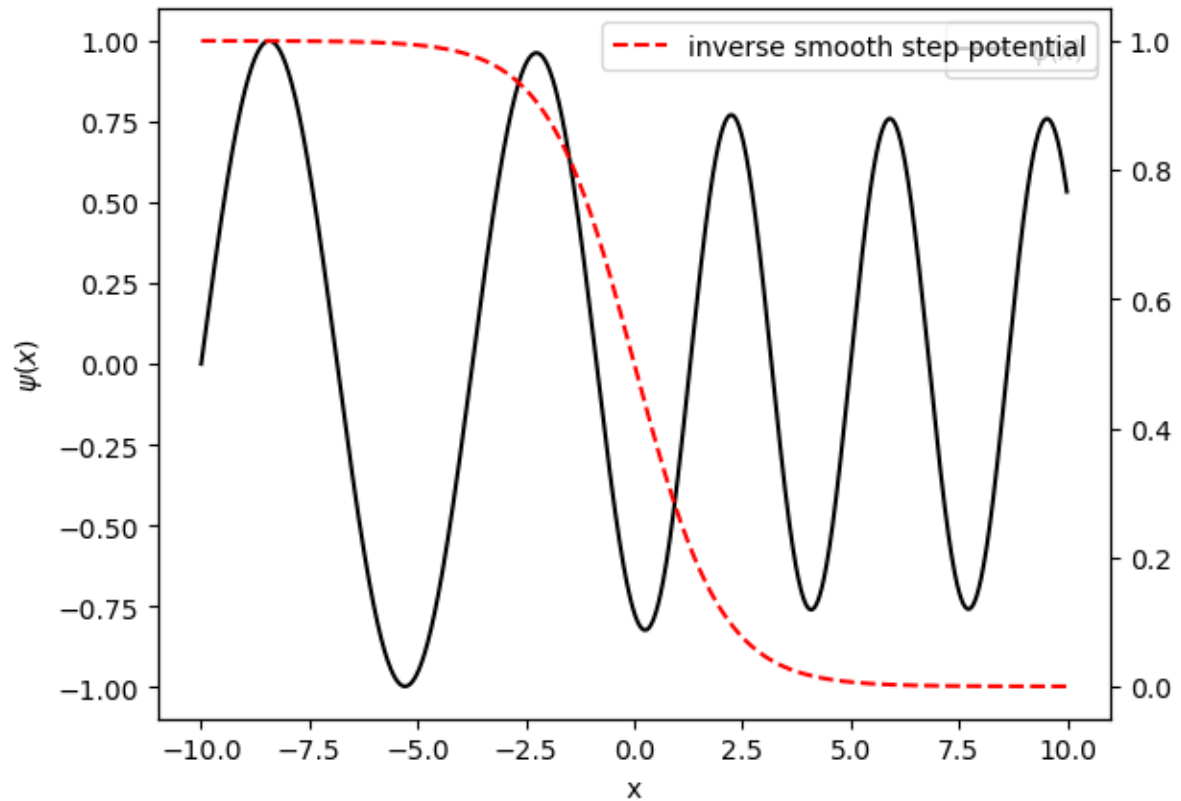
```
# Approximate wave function
E = 1.5
psis = np.zeros(len(xs))
```

```
psis = numerov(E, inverse_smooth_step)
pots = np.array([inverse_smooth_step(x) for x in xs])
```

```
fig, ax = plt.subplots()
ax.plot(xs, psis, 'k-', label=r'$\psi(x)$')
ax2 = ax.twinx()
ax2.plot(xs, pots, 'r--', label='inverse smooth step potential')

ax.set_xlabel('x')
ax.set_ylabel(r'$\psi(x)$')
ax.legend()
ax2.legend()
```

```
<matplotlib.legend.Legend at 0x186b61c1f90>
```



```
# Initial wave function
psi_0, psi_1 = 0, 1.0*epsilon
```

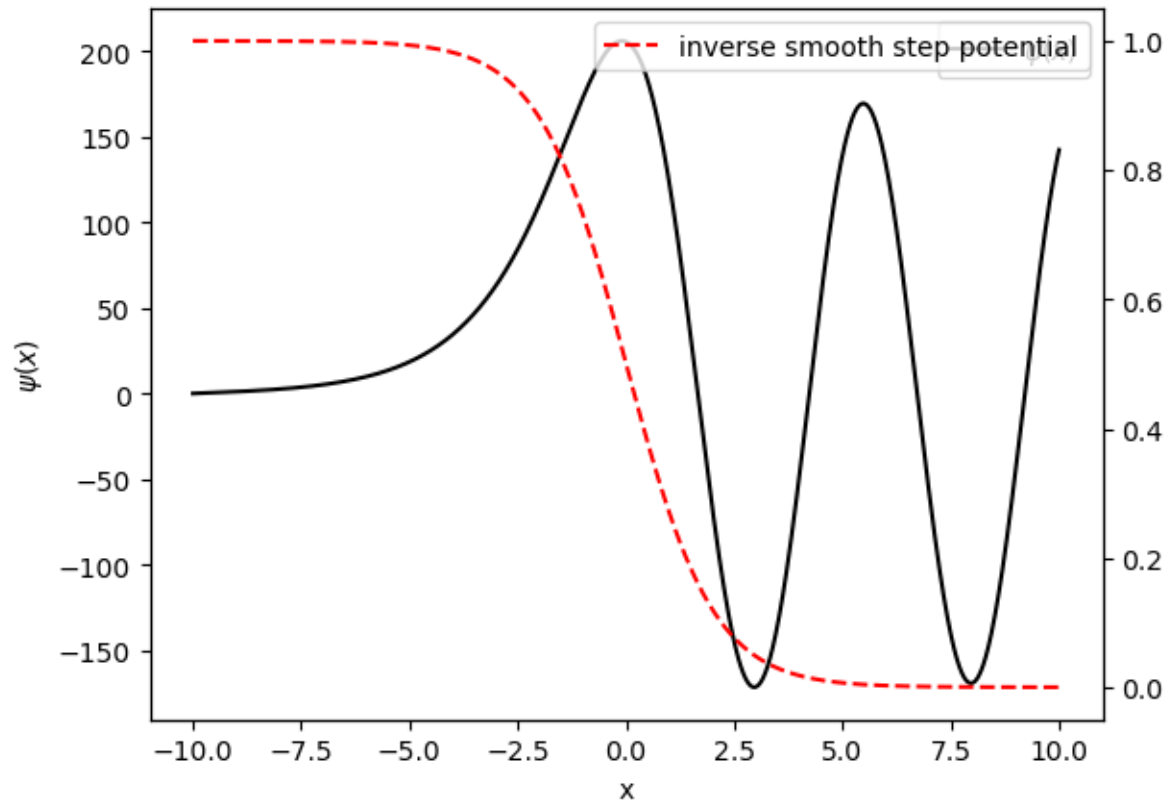
```
# Approximate wave function
E = 0.8
psis = np.zeros(len(xs))
```

```
psis = numerov(E, inverse_smooth_step)
pots = np.array([inverse_smooth_step(x) for x in xs])
```

```
fig, ax = plt.subplots()
ax.plot(xs, psis, 'k-', label=r'$\psi(x)$')
ax2 = ax.twinx()
ax2.plot(xs, pots, 'r--', label='inverse smooth step potential')

ax.set_xlabel('x')
ax.set_ylabel(r'$\psi(x)$')
ax.legend()
ax2.legend()
```

```
<matplotlib.legend.Legend at 0x186b63c3750>
```



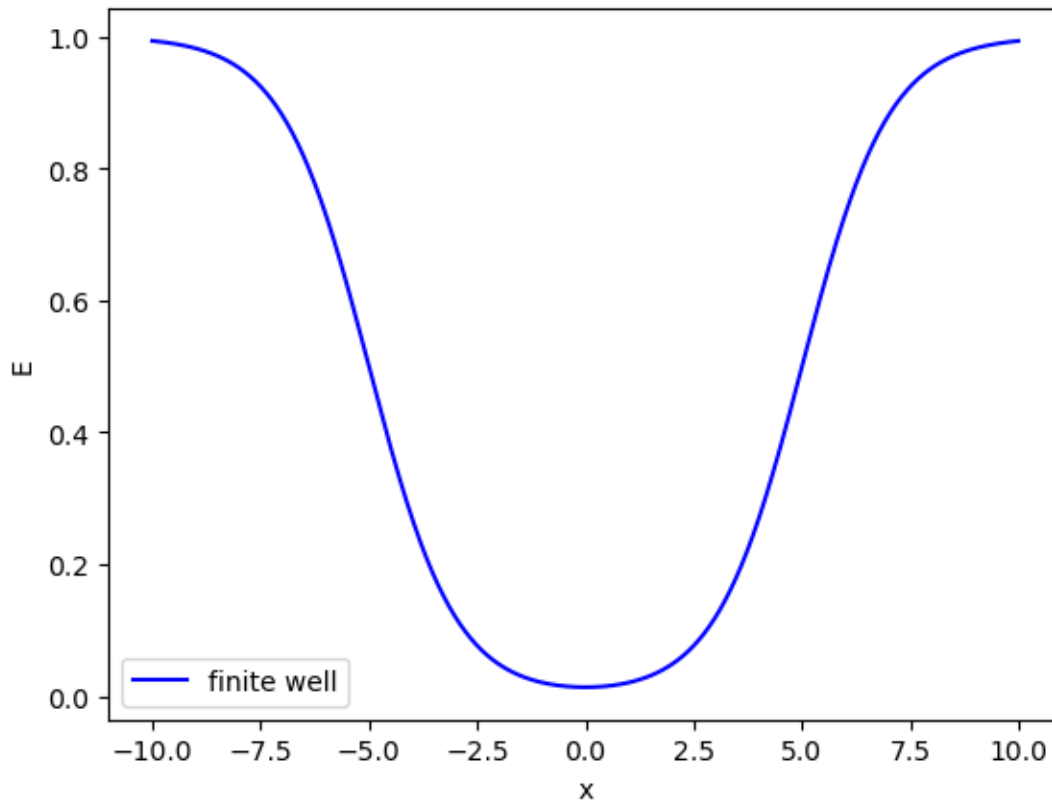
```
# globals
well_width = 10.0
```

```
def finite_well(x):
    halfwidth = well_width/2.0
    return inverse_smooth_step(x+halfwidth) + smooth_step(x-halfwidth)
```

```
pots = np.array([finite_well(x) for x in xs])
```

```
fig, ax = plt.subplots()
ax.plot(xs, pots, 'b-', label='finite well')
ax.set_xlabel('x')
ax.set_ylabel('E')
ax.legend()
```

```
<matplotlib.legend.Legend at 0x186b68d0050>
```



```
# Initial wave function
psi_0, psi_1 = 0, 1.0*epsilon
```

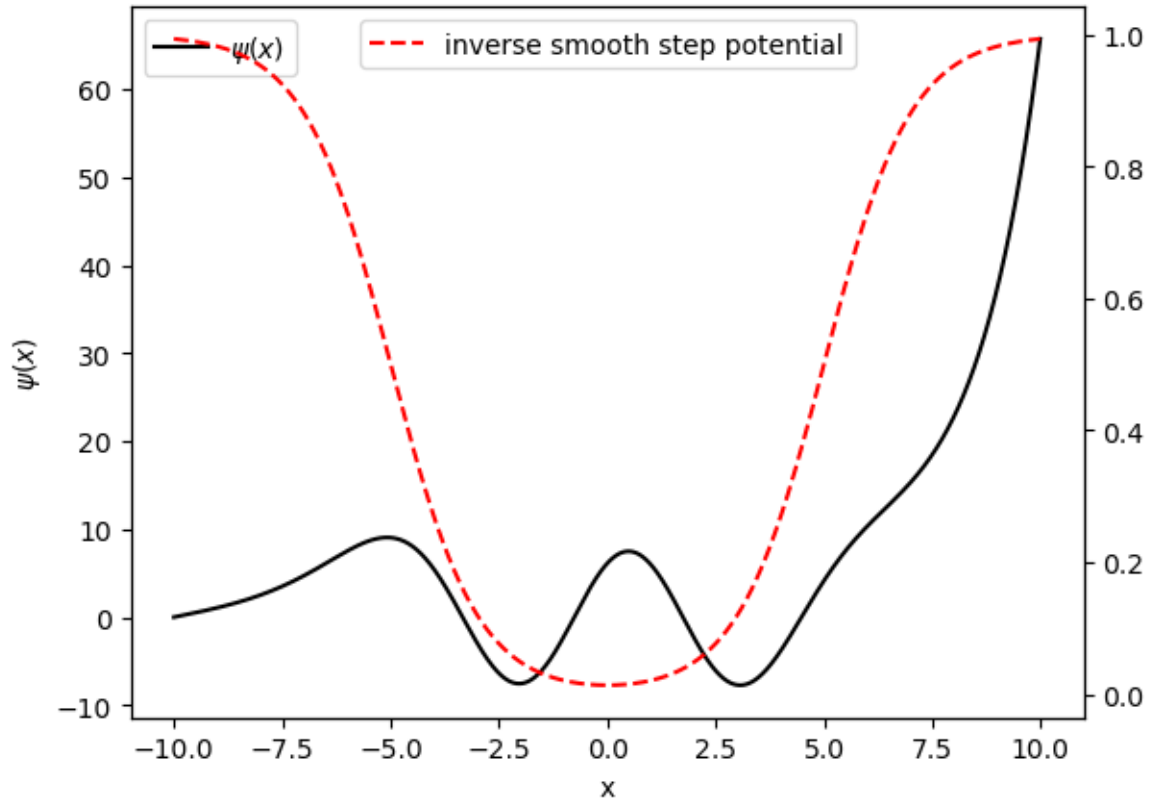
```
# Approximate wave function
E = 0.8
psis = np.zeros(len(xs))
```

```
psis = numerov(E, finite_well)
pots = np.array([finite_well(x) for x in xs])
```

```
fig, ax = plt.subplots()
ax.plot(xs, psis, 'k-', label=r'$\psi(x)$')
ax2 = ax.twinx()
ax2.plot(xs, pots, 'r--', label='inverse smooth step potential')

ax.set_xlabel('x')
ax.set_ylabel(r'$\psi(x)$')
ax.legend()
ax2.legend()
```

```
<matplotlib.legend.Legend at 0x186b697c550>
```



3.5.5 Quantum structures that host scattering states

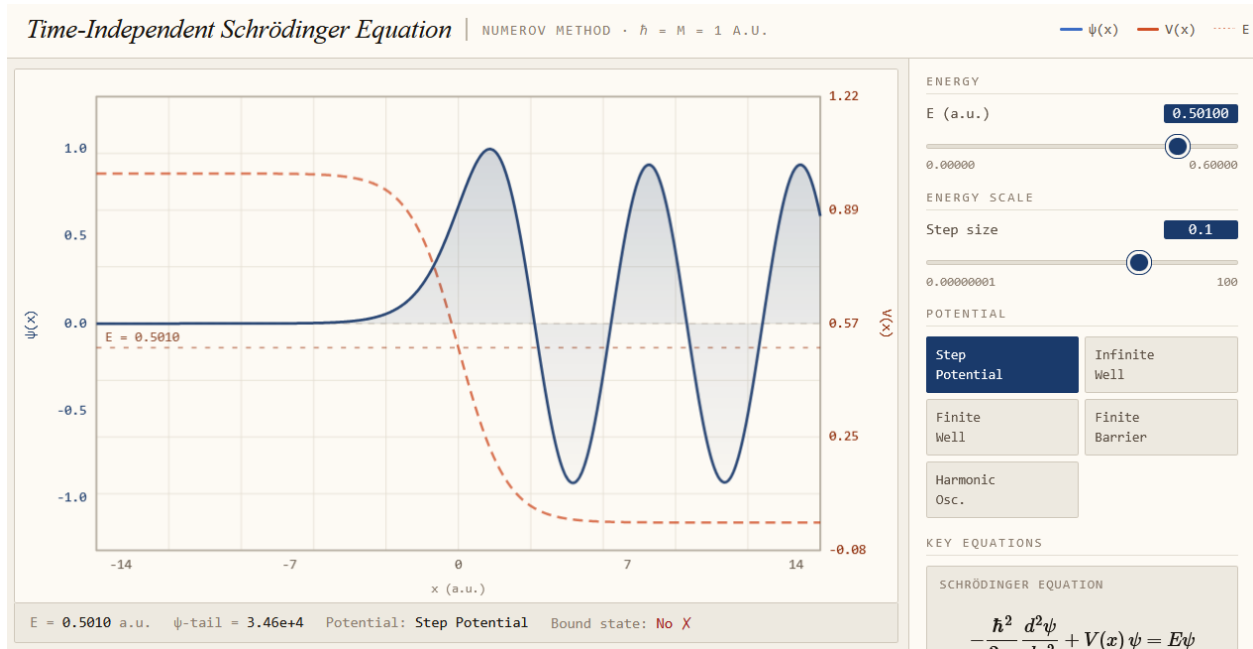
- Step potential
- Potential barrier

3.5.6 Step potential

Depending on the comparison of energy of particle with the step height, there are two cases to consider

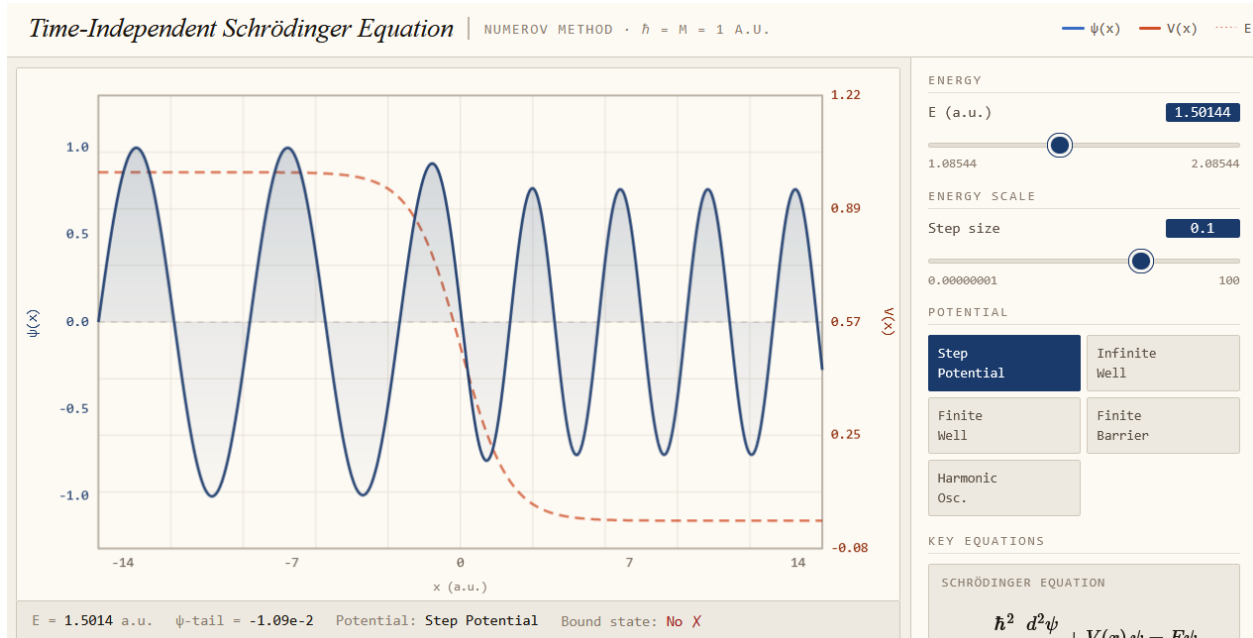
3.5.7 Quantum penetration $E < V_0$

For this case, in the step region, the kinetic energy is less than zero, so the wavefunction has exponential nature. Outside the step region, the wavefunction has oscillatory nature. Since the wavefunction penetrates into the classically forbidden region, this effect is called **quantum penetration**.



3.5.8 Quantum scattering $E > V_0$

For this case, in the step region, the kinetic energy is greater than zero, so the wavefunction has oscillatory nature. Outside the step region, the wavefunction has oscillatory nature. However, the wavevector in both the regions are different; infact wavevector in step region is less than wave vector in outside region.



3.6 Quantum structures that host bound particle

- Infinite potential well
- Finite potential well
- Harmonic oscillator

3.6.1 Infinite potential

Potential energy function

$$V(x) = \begin{cases} 0 & 0 < x < a \\ \infty & \text{otherwise} \end{cases}$$

where a is the width of the box.

Analytical solution

The analytical solution for TISE of particle in an infinite potential well has pedagogical importance.

Let us consider the wavefunction to have the form

$$\psi(x) = A \sin kx + B \cos kx$$

At the boundary $x = 0$, the wavefunction must vanish so that

$$\psi(0) = 0 \Leftrightarrow B = 0$$

Similarly, at the other boundary $x = a$, the wavefunction must vanish so that

$$\psi(a) = 0 \Leftrightarrow \sin ka = 0 \Leftrightarrow k = \frac{n\pi}{a}, \quad n = 1, 2, 3, \dots$$

For each k , there is an associated quantum number n , so that $k \rightarrow k_n$, and the wavefunction is given by

$$\psi_n(x) = A \sin \frac{n\pi x}{a}$$

Upon normalizing the wavefunction, using

$$\int_0^a \psi_n^*(x) \psi_n(x) dx = 1 \Leftrightarrow A = \sqrt{\frac{2}{a}}$$

Therefore, the wavefunction is given by

$$\psi_n(x) = \sqrt{\frac{2}{a}} \sin \frac{n\pi x}{a}$$

Energy eigen values

The energy of the particle is given by

$$E = T + V$$

Since the potential energy V is zero

$$E = T$$

But kinetic energy is given by

$$E_n = \frac{\hbar^2 k_n^2}{2m} = \frac{n^2 \hbar^2 \pi^2}{2ma^2} = \frac{n^2 \pi^2}{a^2}$$

3.7 Quantum confinement

- Quantum dots
- Quantum wells

#TODO Need to work on this notebook.

QUANTUM COMPUTING

Idea map

- *Classical computation*
- *Quantum computation*
- *Classical computation on a quantum computer*
- *Quantum computation on a classical computer*

4.1 Classical computation

Circuit model of classical computation

- binary states
- classical gates
- classical circuit
- classical algorithm

4.1.1 Gates

- NOT
- AND
- OR
- XOR
- NAND
- NOR

4.1.2 Universal logic gate

- NAND gate is a universal gate.

4.2 Quantum computation

Circuit model of quantum computation

- Distinguishable computational basis states
- Universal set of quantum gates
- Preparation of initial state
- Quantum measurement
- Quantum algorithm

4.2.1 Quantum gates

- X
- Y
- Z
- H
- CNOT
- Toffoli

CNOT is a generalization of classical XOR gate

4.2.2 Universal set of quantum logic gates

- Any multiple qubit gate can be composed of CNOT plus single qubit gates
- Toffoli gate based quantum computation

4.3 Classical simulation of quantum computation

We will learn this using Qiskit.

4.4 Quantum simulation of classical computation

A Toffoli gate can be used to simulate NAND gate.

Qiskit is a open source Python package developed by IBM for simulating a quantum computation on a classical computer.

5.1 Installation

- Install miniconda
- Qiskit environment
 - Setup qiskit environment
 - Test qiskit setup

5.2 Qiskit objects

- *QuantumCircuit*
- *Statevector*
- *Operator*
- *Registers*
- *Visualization tools*
- *Multi qubit circuits*

5.3 QuantumCircuit - setup single qubit circuits

In this notebook, we begin our journey into quantum computation.

We will learn:

- What is a qubit?
- What is a quantum circuit?
- How to represent quantum and classical registers
- How to apply basic quantum gates

We will also start understanding how programming concepts (objects) relate to physics.

5.3.1 What is a Qubit?

A classical bit can be either:

- 0
- 1

A quantum bit (qubit) can be:

- $|0\rangle$
- $|1\rangle$
- A combination (superposition) of both

In this notebook, we will not yet compute the state explicitly — we will focus on building circuits.

```
from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister
```

5.3.2 Registers: Organizing Qubits and Bits

In Qiskit:

- QuantumRegister → stores qubits
- ClassicalRegister → stores measurement results

Think of them as containers.

```
# Create a quantum register with 1 qubit
q = QuantumRegister(1, name='q')

# Create a classical register with 1 bit
c = ClassicalRegister(1, name='c')

# Create a quantum circuit using both registers
qc = QuantumCircuit(q, c)

qc
```

```
<qiskit.circuit.quantumcircuit.QuantumCircuit at 0x1eca0bbd840>
```

5.3.3 Observation

- The circuit is currently empty.
- It contains qubits and classical bits, but no operations.

Important: A quantum circuit stores **operations**, not the quantum state itself.

5.3.4 Applying Quantum Gates

We now apply operations (gates) to qubits.

Some basic gates:

- X gate $\rightarrow$ flips $|0\rangle$ to $|1\rangle$
- H gate $\rightarrow$ creates superposition

```
# Apply X gate
qc.x(q[0])

qc
```

```
<qiskit.circuit.quantumcircuit.QuantumCircuit at 0x1eca0bbd840>
```

Try replacing X with H

What do you expect will happen?

(Hint: H creates superposition)

```
# Exercise:
# Replace X with H gate

qc2 = QuantumCircuit(q, c)

# YOUR CODE HERE

qc2
```

```
<qiskit.circuit.quantumcircuit.QuantumCircuit at 0x1eca0bbdff0>
```

5.3.5 Measurement

To extract information, we measure the qubit.

Measurement maps: quantum $\rightarrow$ classical

```
qc.measure(q[0], c[0])

qc
```

```
<qiskit.circuit.quantumcircuit.QuantumCircuit at 0x1eca0bbd840>
```

5.3.6 Key Concept

- QuantumRegister → holds qubits
- ClassicalRegister → holds measurement outcomes
- QuantumCircuit → stores operations (gates + measurement)

The circuit is like a **program**, not the result.

5.3.7 Exercise 1

Create a circuit with:

- 1 qubit
- 1 classical bit
- Apply H gate
- Then measure

Draw the circuit.

YOUR SOLUTION HERE

5.3.8 Exercise 2

Create a circuit:

1. Apply X gate
2. Then apply H gate
3. Measure

Question: Does the order of gates matter?

YOUR SOLUTION HERE

5.3.9 Conceptual Questions

1. Does the quantum circuit store the state of the system?
2. What is the role of the classical register?
3. What happens if we do not include measurement?

Write your answers below.

Your Answers:

- 1.
- 2.
- 3.

5.4 Statevector

In the previous notebook, we built quantum circuits.

But an important question remains:

What is the **state** of the qubit after applying gates?

In this notebook, we will:

- Represent quantum states using vectors
- Apply circuits to states
- Observe how gates change the state

This is where physics and linear algebra enter.

5.4.1 From Circuits to States

A quantum circuit describes operations.

To *see the result*, we use:

☞ Statevector

This object stores the amplitudes of a quantum system.

```
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
```

5.4.2 Initial State

By default, a qubit starts in:

$|0\rangle$

We can create this using Statevector.

```
# Create |0> state
sv = Statevector.from_label('0')

sv
```

```
Statevector([1.+0.j, 0.+0.j],
            dims=(2,))
```

5.4.3 Interpretation

The statevector contains amplitudes:

[1, 0]

Meaning:

- Probability of $|0\rangle = 1$
- Probability of $|1\rangle = 0$

5.4.4 Applying a Circuit to a State

We now:

1. Create a circuit
2. Apply it to the statevector

```
qc = QuantumCircuit(1)
qc.x(0)
```

```
<qiskit.circuit.instructionset.InstructionSet at 0x2d9c0c70820>
```

```
qc.draw('text')
```

```
q: ┌───┐
    │ X │
    └───┘
```

```
# Apply circuit to state
sv_new = sv.evolve(qc)

sv_new
```

```
Statevector([0.+0.j, 1.+0.j],
            dims=(2,))
```

5.4.5 Observation

After applying X gate:

$|0\rangle \rightarrow |1\rangle$

Statevector becomes: [0, 1]

☞ The circuit changed the state.

5.4.6 Superposition using H Gate

Now let us apply the Hadamard (H) gate.

```
qc_h = QuantumCircuit(1)
qc_h.h(0)
sv_h = sv.evolve(qc_h)

sv_h
```

```
Statevector([0.70710678+0.j, 0.70710678+0.j],
            dims=(2,))
```

5.4.7 Interpretation

The result is approximately:

$[1/\sqrt{2}, 1/\sqrt{2}]$

Meaning:

- 50% chance of $|0\rangle$
- 50% chance of $|1\rangle$

☞ This is called superposition.

5.4.8 Key Idea

- QuantumCircuit → describes operations
- Statevector → shows the result

The circuit does NOT store the state. The statevector does.

5.4.9 Exercise 1

Create a circuit:

1. Start from $|0\rangle$
2. Apply X gate
3. Then apply H gate

Find the final statevector.

```
# YOUR CODE HERE
```

5.4.10 Exercise 2

Apply H twice:

$H \rightarrow H$

What do you observe?

Is it equal to the original state?

```
qc_test = QuantumCircuit(1)
qc_test.h(0)
qc_test.h(0)

sv_test = sv.evolve(qc_test)

sv_test
```

```
Statevector([1.+0.j, 0.+0.j],
            dims=(2,))
```

5.4.11 Conceptual Questions

1. Does the statevector depend on the circuit?
2. Can two different circuits give the same state?
3. What happens if we never compute the statevector?

Think and write your answers.

Your Answers:

- 1.
- 2.
- 3.

5.5 Operator

So far, we have learned:

- QuantumCircuit $\rightarrow$ describes operations
- Statevector $\rightarrow$ shows resulting state
- Measurement $\rightarrow$ gives outcomes

Now we ask:

What is the mathematical form of a quantum circuit?

Answer: A quantum circuit is a matrix (Operator)

In this notebook, we will:

- Represent circuits as matrices

- Apply matrices to quantum states
- Verify consistency with Statevector

5.5.1 From Gates to Matrices

Every quantum gate corresponds to a matrix.

Examples:

- X gate → flips states
- H gate → creates superposition

A circuit is a combination (multiplication) of these matrices.

```
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector, Operator
```

5.5.2 Creating a Simple Circuit

```
qc = QuantumCircuit(1)
qc.x(0)

qc
```

```
<qiskit.circuit.quantumcircuit.QuantumCircuit at 0x1934b930550>
```

```
qc.draw('mpl')
```



5.5.3 Converting Circuit to Operator

```
op = Operator(qc)

op
```

```
Operator([[0.+0.j, 1.+0.j],
          [1.+0.j, 0.+0.j]],
         input_dims=(2,), output_dims=(2,))
```

5.5.4 Interpretation

The operator is a matrix:

`[[0, 1], [1, 0]]`

Let us write the X operator neatly using LaTeX notation

$$\tilde{X} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

This is exactly the X gate matrix.

So: QuantumCircuit $\rightarrow$ Operator (matrix)

5.5.5 Applying Operator to a State

```
# Initial state |0>
sv = Statevector.from_label('0')

# Apply operator
sv_new = sv.evolve(op)

sv_new
```

```
Statevector([0.+0.j, 1.+0.j],
            dims=(2,))
```

```
sv_new_2 = sv.evolve(qc)

sv_new_2
```

```
Statevector([0.+0.j, 1.+0.j],
            dims=(2,))
```

5.5.6 Key Idea

Applying:

- circuit to statevector
- operator to statevector

should give the SAME result.

```
# Using circuit
sv1 = sv.evolve(qc)

# Using operator
sv2 = sv.evolve(op)

sv1, sv2
```

```
(Statevector([0.+0.j, 1.+0.j],
             dims=(2,)),
 Statevector([0.+0.j, 1.+0.j],
             dims=(2,)))
```

5.5.7 Observation

Both results are identical.

☞ Circuit = Operator (mathematically)

5.5.8 Combining Gates

Let us apply two gates: H followed by X

```
qc2 = QuantumCircuit(1)
qc2.h(0)
qc2.x(0)

op2 = Operator(qc2)

op2
```

```
Operator([[ 0.70710678+0.j, -0.70710678+0.j],
          [ 0.70710678+0.j,  0.70710678+0.j]],
         input_dims=(2,), output_dims=(2,))
```

```
# matrix multiplication
import numpy as np
matrix_h = np.array([[1/np.sqrt(2), 1/np.sqrt(2)], [1/np.sqrt(2), -1.0*1/np.sqrt(2)]])
matrix_x = np.array([[0.0, 1.0], [1.0, 0.0]])

matrix_qc = matrix_x @ matrix_h
matrix_qc

np.allclose(op2.to_matrix(), matrix_qc)
```

```
True
```

```
op2.to_matrix()
```

```
array([[ 0.70710678+0.j, -0.70710678+0.j],
        [ 0.70710678+0.j,  0.70710678+0.j]])
```

5.5.9 Important Insight

Matrix multiplication follows order:

Last gate applied appears FIRST in matrix multiplication

This is subtle but important.

5.5.10 Exercise 1

Create a circuit:

1. Apply X
2. Then apply H

Convert to operator.

Is it the same as H followed by X?

$$\tilde{H}\tilde{X} = \tilde{X}\tilde{H}$$

$$[H, X] = HX - XH$$

```
# YOUR CODE HERE
```

5.5.11 Exercise 2

Verify the statement:

Operator(qc) applied to statevector is the same as the statevector evolved by qc.

Try for different circuits.

```
# YOUR CODE HERE
```

```
qc = QuantumCircuit(1)
qc.x(0)
qc.h(0)
```

```
op = Operator(qc)
op
```

```
Operator([[ 0.70710678+0.j,  0.70710678+0.j],
          [-0.70710678+0.j,  0.70710678+0.j]],
         input_dims=(2,), output_dims=(2,))
```

```
sv = Statevector.from_label('0')
sv_circuit = sv.evolve(qc)
sv_circuit
```

```
Statevector([ 0.70710678+0.j, -0.70710678+0.j],
            dims=(2,))
```

```
sv_operator = sv.evolve(op)
```

```
sv_operator
```

```
Statevector([ 0.70710678+0.j, -0.70710678+0.j],  
            dims=(2,))
```

```
sv_circuit==sv_operator
```

```
True
```

5.5.12 Conceptual Questions

1. Why must quantum operators be reversible?
2. What would happen if the matrix is not unitary?
3. Can two different circuits have the same operator?

Explain your reasoning.

Your Answers:

1. quantum operators are reversible so that the norm is preserved. It can be considered as a conservation of probability law similar to conservation of energy or momentum.
2. If the matrix is not unitary, then the probability may not be conserved.
3. Yes, it is entirely possible.

5.6 Registers - Measurement in Quantum Computation

So far, we have:

- Built quantum circuits
- Observed quantum states using Statevector

But in real experiments, we do NOT see amplitudes.

We only observe measurement outcomes.

In this notebook, we will:

- Understand measurement
- Use classical registers
- Connect probabilities to outcomes

5.6.1 Quantum vs Classical Information

- Quantum state → hidden (not directly observable)
- Measurement → gives classical result (0 or 1)

This is why we need: ClassicalRegister

```
from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister
from qiskit.quantum_info import Statevector
```

5.6.2 Creating a Measurable Circuit

```
q = QuantumRegister(1, 'q')
c = ClassicalRegister(1, 'c')

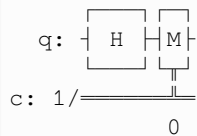
qc = QuantumCircuit(q, c)

qc.h(q[0])    # create superposition
qc.measure(q[0], c[0])

qc
```

```
<qiskit.circuit.quantumcircuit.QuantumCircuit at 0x1cc8c6c19c0>
```

```
qc.draw('text')
```



5.6.3 What Do We Expect?

After H gate:

State = $(|0\rangle + |1\rangle)/\sqrt{2}$

Measurement outcomes:

- 0 with probability 0.5
- 1 with probability 0.5

But a single measurement gives ONLY one result.

5.6.4 Simulating Measurement Using Statevector

We can extract probabilities from the statevector.

```
# Create statevector before measurement
qc_no_measure = QuantumCircuit(1)
qc_no_measure.h(0)

sv = Statevector.from_label('0')
sv_final = sv.evolve(qc_no_measure)

# Get probabilities
probs = sv_final.probabilities()

probs
```

```
array([0.5, 0.5])
```

5.6.5 Interpretation

The output shows probabilities:

```
[0.5, 0.5]
```

This predicts measurement statistics.

5.6.6 Repeated Measurement (Conceptual)

In real experiments:

- We repeat the circuit many times
- Collect outcomes

This gives a distribution of results.

```
import numpy as np

# Simulate 100 measurements
samples = np.random.choice([0, 1], size=100, p=probs)

# Count results
unique, counts = np.unique(samples, return_counts=True)

dict(zip(unique, counts))
```

```
{np.int64(0): np.int64(51), np.int64(1): np.int64(49)}
```

5.6.7 Observation

- Results are approximately balanced
- Not exactly equal (randomness!)

This mimics real quantum experiments.

5.6.8 Key Concepts

- Statevector → gives probabilities
- Measurement → gives one outcome
- ClassicalRegister → stores outcome

Quantum → Classical transition happens during measurement.

5.6.9 Exercise 1

Create a circuit:

1. Apply X gate
2. Measure

What are the expected probabilities?

Verify using Statevector.

```
# YOUR CODE HERE
```

5.6.10 Exercise 2

Create a circuit:

1. Apply H
2. Apply H again
3. Measure

What do you expect the result to be?

```
# YOUR CODE HERE
```

5.6.11 Conceptual Questions

1. Why can't we directly observe the statevector?
2. Why do we need repeated measurements?
3. Does measurement change the quantum state?

Explain in your own words.

Your Answers:

- 1.
- 2.
- 3.

5.7 Visualization of Quantum States

So far, we have:

- Built circuits
- Computed statevectors
- Understood operators

But quantum states are abstract.

Visualization helps us build intuition.

In this notebook, we will:

- Visualize quantum states on the Bloch sphere
- Plot measurement probabilities
- Connect visuals to physical meaning

5.7.1 Required Tools

We use Qiskit visualization functions.

```
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_bloch_multivector, plot_histogram
```

5.7.2 The Bloch Sphere

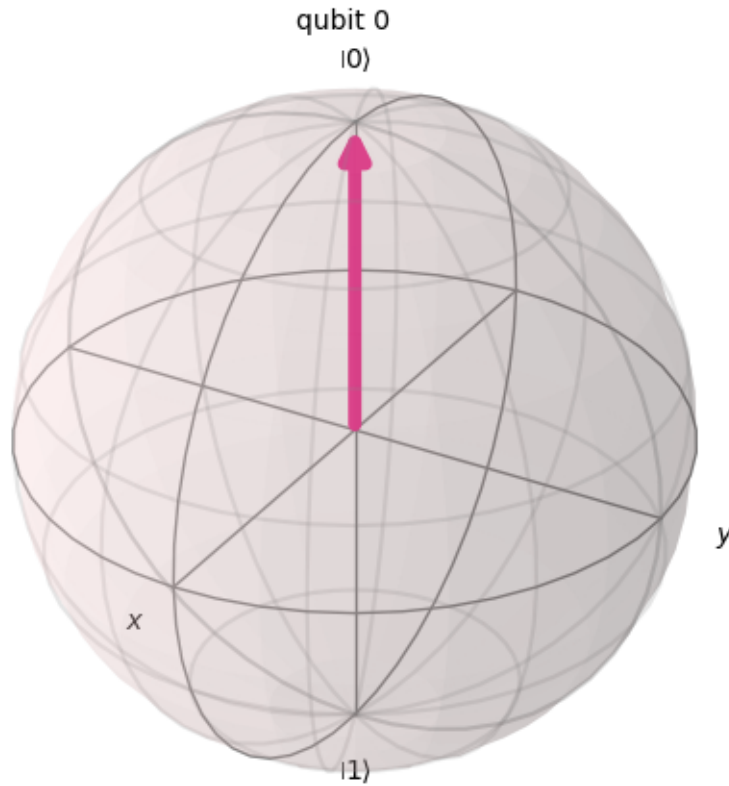
A single qubit state can be represented as a point on a sphere.

Key states:

- $|0\rangle \rightarrow$ north pole
- $|1\rangle \rightarrow$ south pole
- superpositions $\rightarrow$ points in between

```
# |0> state
sv0 = Statevector.from_label('0')

plot_bloch_multivector(sv0)
```



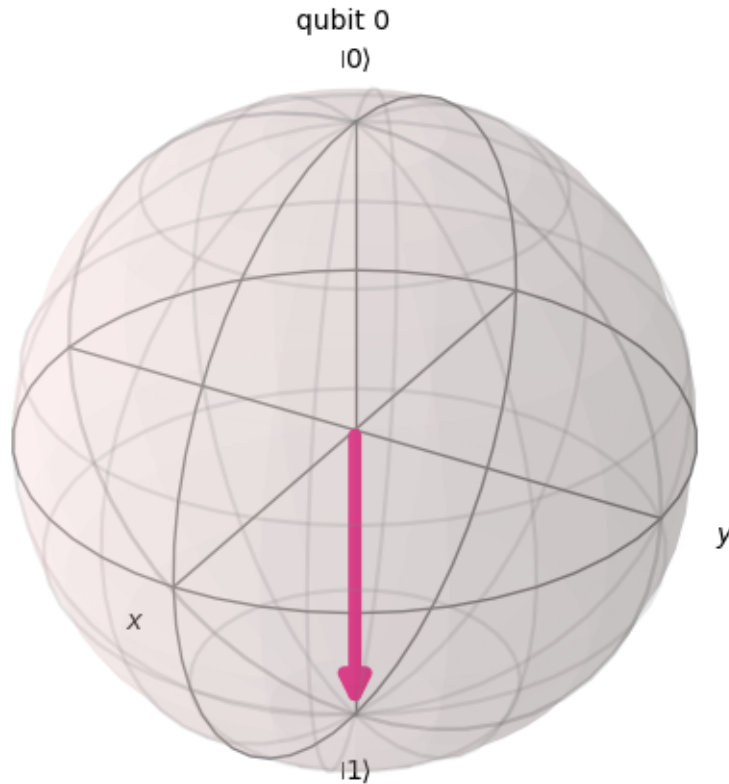
5.7.3 Observation

The vector points along the +Z direction.

This represents $|0\rangle$

```
# |1> state
sv1 = Statevector.from_label('1')

plot_bloch_multivector(sv1)
```



5.7.4 Observation

The vector points along the -Z direction.

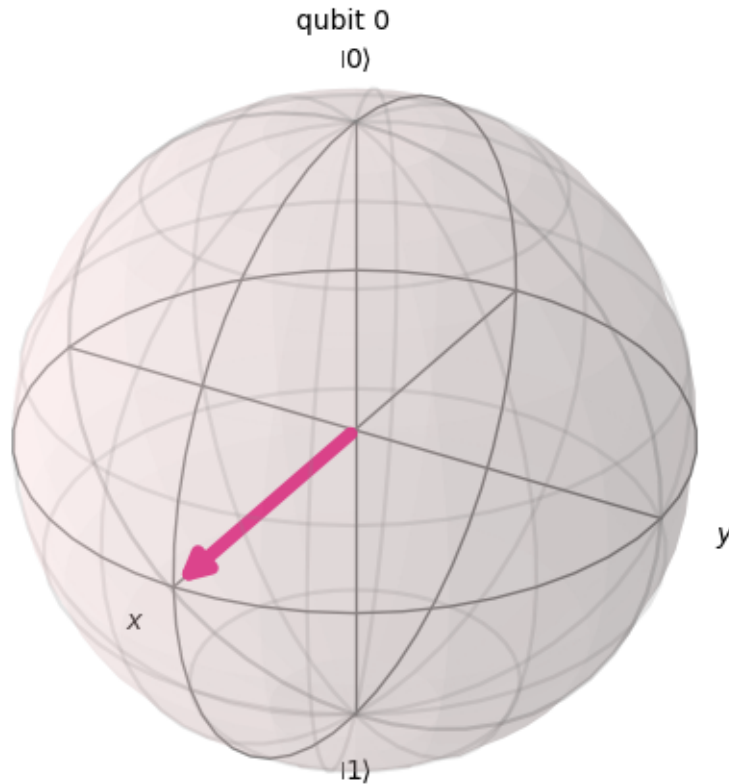
This represents $|1\rangle$

5.7.5 Superposition (H Gate)

```
qc = QuantumCircuit(1)
qc.h(0)

sv_h = Statevector.from_label('0').evolve(qc)

plot_bloch_multivector(sv_h)
```



5.7.6 Interpretation

The vector lies on the equator.

Equal probability of $|0\rangle$ and $|1\rangle$

This is a balanced superposition.

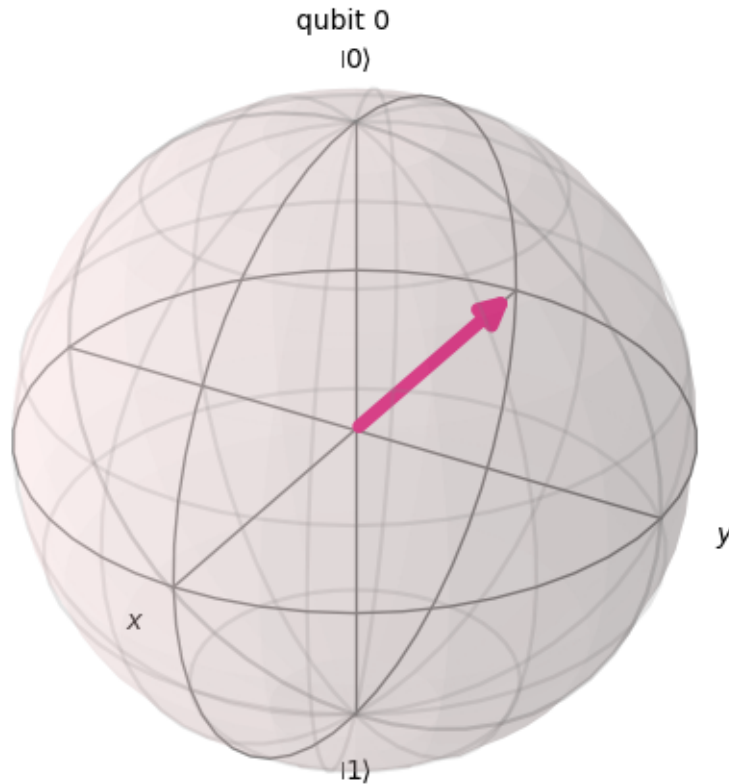
5.7.7 Phase Matters

Different states can have same probabilities but different phases.

Let us apply Z gate after H.

```
qc2 = QuantumCircuit(1)
qc2.h(0)
qc2.z(0)

sv_hz = Statevector.from_label('0').evolve(qc2)
plot_bloch_multivector(sv_hz)
```



5.7.8 Key Insight

- Probabilities remain the same
- Direction changes on Bloch sphere

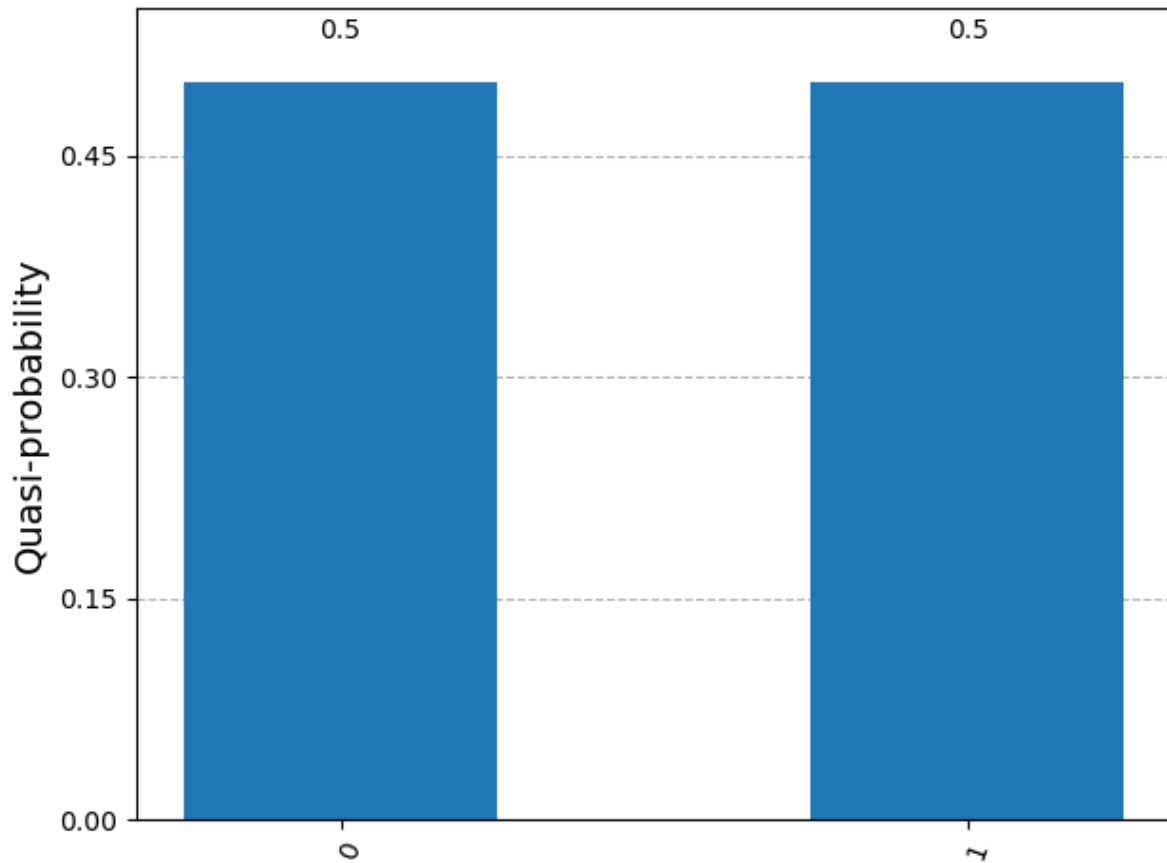
Phase is invisible in measurement but visible in state geometry

5.7.9 Measurement Visualization (Histogram)

We now connect state $\rightarrow$ probabilities $\rightarrow$ outcomes

```
# Probabilities from H state
probs = sv_h.proBABILITIES_dict()

plot_histogram(probs)
```



5.7.10 Interpretation

- Bars represent probabilities
- Heights correspond to likelihood of outcomes

This predicts measurement results

5.7.11 Exercise 1

Create and visualize:

1. Apply X gate to $|0\rangle$
2. Plot Bloch sphere

Where is the vector located?

```
# YOUR CODE HERE
```

5.7.12 Exercise 2

Create:

1. $H \rightarrow Z \rightarrow H$

Plot Bloch sphere.

What do you observe?

```
# YOUR CODE HERE
```

5.7.13 Conceptual Questions

1. Can two different Bloch vectors give same measurement probabilities?
2. Why is phase not visible in measurement?
3. What does the equator of the Bloch sphere represent?

Explain clearly.

Your Answers:

- 1.
- 2.
- 3.

5.8 Setup multi qubit circuits

Let us work with two qubit circuit.

The important two qubit gates are

- CNOT

5.8.1 Learning Objectives

- Understand tensor product structure of multi-qubit systems
- Construct multi-qubit circuits using Qiskit
- Study entanglement (Bell states)
- Analyze circuits using Statevector and Operator
- Visualize measurement outcomes

```
from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister
from qiskit.quantum_info import Statevector, Operator
import numpy as np
import matplotlib.pyplot as plt
```

5.8.2 Multi-Qubit State Representation

A single qubit:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

Two qubits:

$$|\psi\rangle = \alpha|00\rangle + \beta|01\rangle + \gamma|10\rangle + \delta|11\rangle$$

5.8.3 Multi-Qubit State Representation

```
# Two-qubit basis states
basis_00 = Statevector.from_label('00')
basis_01 = Statevector.from_label('01')
basis_10 = Statevector.from_label('10')
basis_11 = Statevector.from_label('11')

print(basis_00)
```

```
Statevector([1.+0.j, 0.+0.j, 0.+0.j, 0.+0.j],
            dims=(2, 2))
```

5.8.4 Tensor Product Concept

```
# |0> ⊗ |1> = |01>
psi = Statevector.from_label('0').tensor(Statevector.from_label('1'))
print(psi)
```

```
Statevector([0.+0.j, 1.+0.j, 0.+0.j, 0.+0.j],
            dims=(2, 2))
```

5.8.5 Creating a 2-Qubit Circuit

```
qr = QuantumRegister(2)
cr = ClassicalRegister(2)
qc = QuantumCircuit(qr, cr)

qc.draw('mpl')
```

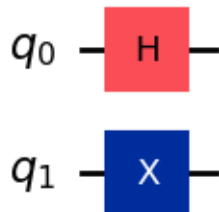
$q0_0$ —
 $q0_1$ —
 $c0$ ~~$\frac{2}{\text{—}}$~~

5.8.6 Single-Qubit Gates on Multi-Qubit Systems

```
qc = QuantumCircuit(2)

qc.h(0) # Apply Hadamard to qubit 0
qc.x(1) # Apply NOT to qubit 1

qc.draw('mpl')
```

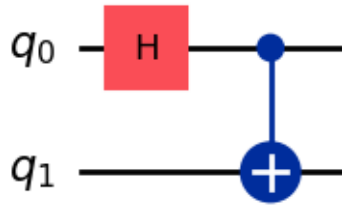


5.8.7 Controlled Gates (CNOT)

```
qc = QuantumCircuit(2)

qc.h(0)
qc.cx(0, 1)

qc.draw('mpl')
```



5.8.8 Bell State Creation (Entanglement)

$$|\Phi+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

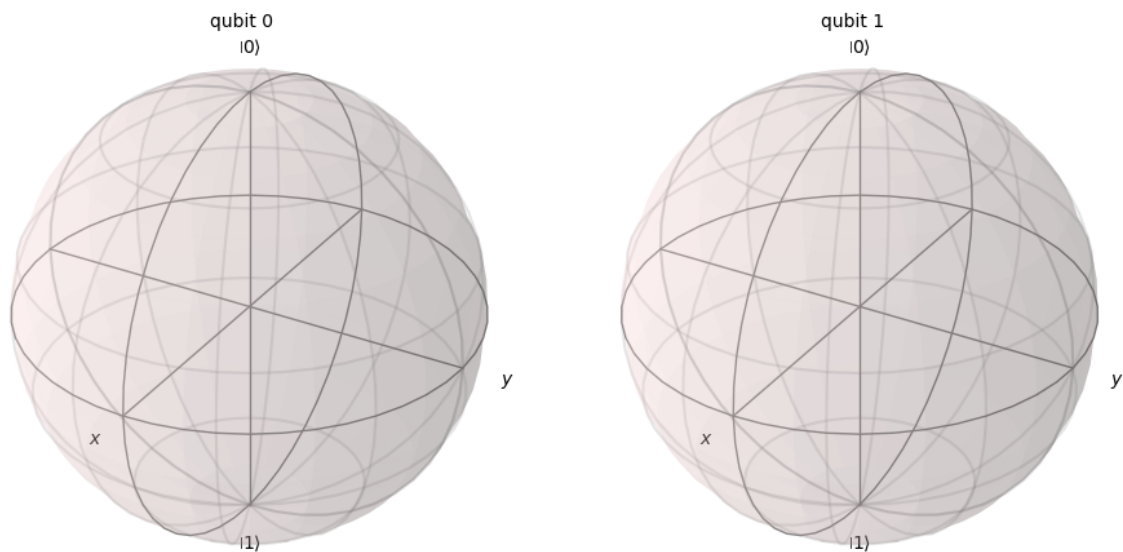
```
qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0, 1)

state = Statevector.from_instruction(qc)
print(state)
```

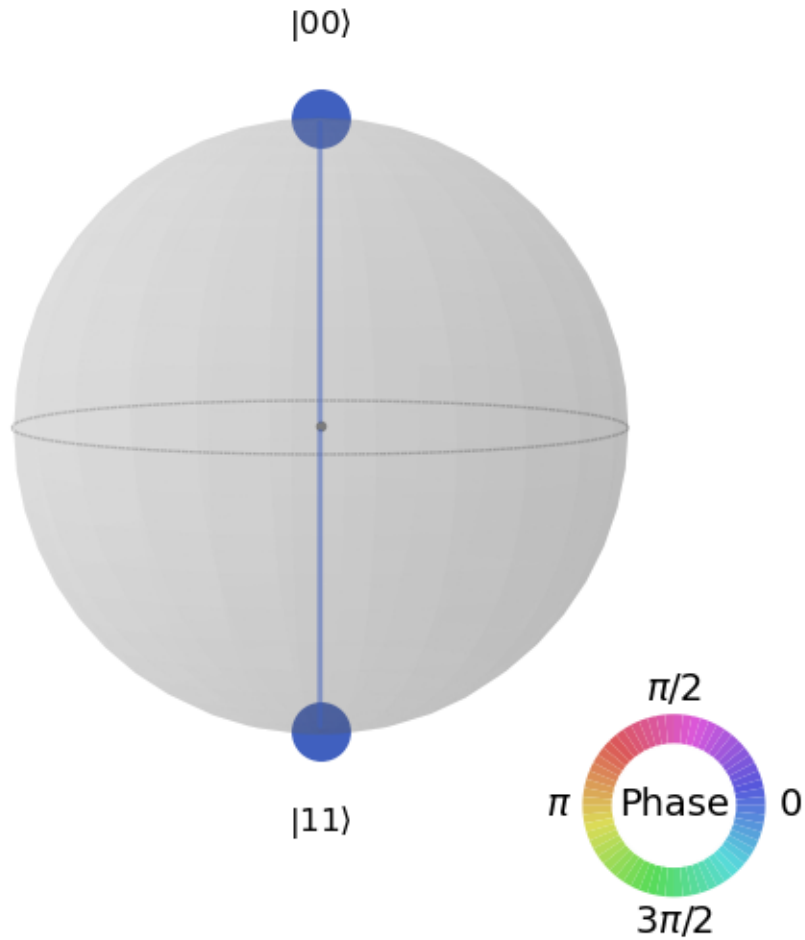
```
Statevector([0.70710678+0.j, 0.          +0.j, 0.          +0.j,
             0.70710678+0.j],
            dims=(2, 2))
```

5.8.9 Visualizing Statevector

```
state.draw('bloch')
```



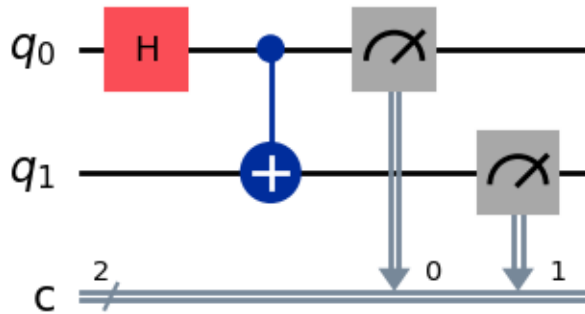
```
state.draw('qsphere')
```



5.8.10 Measurement

```
qc = QuantumCircuit(2, 2)
qc.h(0)
qc.cx(0, 1)

qc.measure([0,1], [0,1])
qc.draw('mpl')
```



5.8.11 Simulating measurement outcomes

```
from qiskit_aer import AerSimulator

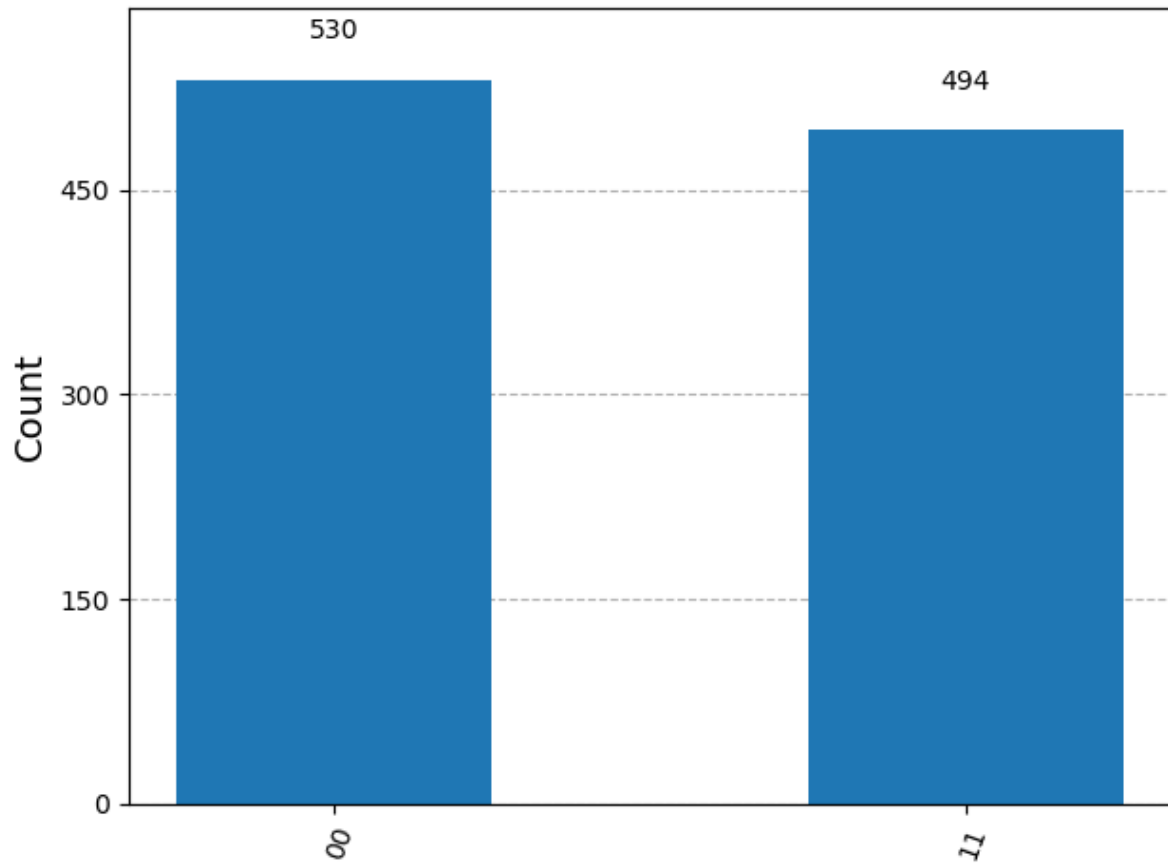
sim = AerSimulator()
result = sim.run(qc, shots=1024).result()
counts = result.get_counts()

print(counts)
```

```
{'00': 530, '11': 494}
```

5.8.12 Plot histogram

```
from qiskit.visualization import plot_histogram
plot_histogram(counts)
```



5.8.13 Operator representation of data

```
qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0,1)

U = Operator(qc)
print(U.data)
```

```
[[ 0.70710678+0.j  0.70710678+0.j  0.          +0.j  0.          +0.j]
 [ 0.          +0.j  0.          +0.j  0.70710678+0.j -0.70710678+0.j]
 [ 0.          +0.j  0.          +0.j  0.70710678+0.j  0.70710678+0.j]
 [ 0.70710678+0.j -0.70710678+0.j  0.          +0.j  0.          +0.j]]
```


HOME WORKS

- Waveket representation of polarization states
- Eigenvalues and eigenvectors of $\tilde{S}_x$
- Unitary transformation
- Dirac notation of Hermitian operator
- Hamiltonian in matrix and Dirac notation
- Differentiation and Integration of functions
- Traveling vs standing waves
- Properties of anti-Hermitian operators

6.1 Waveket representation of polarization states

Write the wave ket representation of polarization states

- circularly,
- elliptically polarized light

6.1.1 Solution

The circularly polarized light can be modelled as two components - the horizontal and vertical components. However, the vertical component is ahead in phase by $\pi/2$ compared to horizontal component.

Also, the amplitude weightage is same. Therefore,

$$\begin{aligned} |C_+\rangle &= \frac{1}{\sqrt{2}}|H\rangle + \exp(i\frac{\pi}{2})\frac{1}{\sqrt{2}}|V\rangle \\ &= \frac{1}{\sqrt{2}}(|H\rangle + i|V\rangle) \end{aligned} \quad (6.1)$$

This is a right circularly polarized light. Similarly, we have left circularly polarized light...

$$\begin{aligned} |C_-\rangle &= \frac{1}{\sqrt{2}}|H\rangle + \exp(-i\frac{\pi}{2})\frac{1}{\sqrt{2}}|V\rangle \\ &= \frac{1}{\sqrt{2}}(|H\rangle - i|V\rangle) \end{aligned} \quad (6.3)$$

In the matrix representation,

$$|C_{\pm}\rangle \begin{pmatrix} 1 \\ \pm i \end{pmatrix}$$

For the elliptically polarized light, the amplitudes are different.

6.2 Eigen values and eigen kets of $\tilde{S}_x$

The action of $\tilde{S}_x$ is given by

$$\begin{aligned}\tilde{S}_x 0 &= \frac{\hbar}{2} 1 \\ \tilde{S}_x 1 &= \frac{\hbar}{2} 0\end{aligned}$$

Find the eigenvalues and eigenkets of $\tilde{S}_x$.

6.2.1 Solution

The matrix representation of the computational basis $\{0, 1\}$ is given by

$$0 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad 1 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

In this basis, the action on the basis is given by

$$\tilde{S}_x 0 = \frac{\hbar}{2} \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \quad \tilde{S}_x 1 = \frac{\hbar}{2} \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

Therefore, the matrix representation of $\tilde{S}_x$ is given by

$$\tilde{S}_x = \frac{\hbar}{2} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

6.2.2 Eigen values

The eigen values are given by solutions to the determinant equation

$$\begin{vmatrix} -\lambda & 1 \\ 1 & -\lambda \end{vmatrix} = 0.$$

$$\lambda^2 = 1 \Leftrightarrow \lambda = \pm 1$$

6.2.3 Eigen kets

Case (I): $\lambda = +1$

$$\begin{pmatrix} -1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix} = 0$$

$$\alpha = \beta$$

So the eigen ket is parallel to

$$+1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

Since the eigen ket should have norm equal to one,

$$+1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

Case (II): $\lambda = -1$

$$\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix} = 0$$

$$\alpha = -\beta$$

So the normalized eigen ket is

$$-1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

6.3 Unitary transformation

This homework is based on the concept of unitary transformation.

Express eigenvectors of $\tilde{S}_z$ in terms of eigenvectors of $\tilde{S}_x$ using

- mathematician's way (inverse transformation way)
- physicist's way (unitary transformation way)

6.3.1 Solution

Let us consider the computational basis $\{0, 1\}$ is the eigen basis of $\tilde{S}_z$.

Therefore, the eigen kets of $\tilde{S}_z$ are

$$+1_z = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad -1_z = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

And the eigenkets of $\tilde{S}_x$ are

$$+1_x = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad -1_x = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

In terms of Dirac notation,

$$+1_x = \frac{1}{\sqrt{2}} (+1_z + -1_z)$$

$$-1_x = \frac{1}{\sqrt{2}} (+1_z - -1_z)$$

Inverse transformation way (mathematician's way)

Let us express the eigenkets of $\tilde{S}_z$ in terms of eigenkets of $\tilde{S}_x$.

$((A) + (B))/2$ gives

$$+1_z = \frac{1}{\sqrt{2}} (+1_x + -1_x)$$

Similarly $((A) - (B))/2$ gives

$$-1_z = \frac{1}{\sqrt{2}} (+1_x - -1_x)$$

Unitary transformation way (physicist's way)

When we found the eigenvalues and eigenkets of the $\tilde{S}_x$ operator, we can consider the matrix formed due to the eigenkets of $\tilde{S}_x$ as the unitary matrix that diagonalizes $\tilde{S}_x$ under the similarity transformation matrix

$$\tilde{S}_x \Rightarrow \tilde{U}^\dagger \tilde{S}_x \tilde{U}$$

```
import numpy as np
from numpy import linalg as LA
```

```
hbar = 1
```

```
sx = hbar/2*np.array([[0, 1], [1, 0]])
```

```
sx
```

```
array([[0. , 0.5],
       [0.5, 0. ]])
```

```
eigenvalues, eigenvectors = LA.eig(sx)
```

```
eigenvalues
```

```
array([ 0.5, -0.5])
```

```
eigenvectors
```

```
array([[ 0.70710678, -0.70710678],
       [ 0.70710678,  0.70710678]])
```

```
unitary = eigenvectors
```

```
unitary
```

```
array([[ 0.70710678, -0.70710678],
       [ 0.70710678,  0.70710678]])
```

```
np.conjugate?
```

```
# check diagonalization

diagonal_matrix = np.conjugate(np.transpose(unitary)) @ sx @ unitary

diagonal_matrix
```

```
array([[ 5.00000000e-01,  1.11022302e-16],
       [ 1.11022302e-16, -5.00000000e-01]])
```

```
# unitary transformation

ket_0_sz = np.transpose(np.array([1, 0]))
ket_1_sz = np.transpose(np.array([0, 1]))

ket_0_sz_prime = unitary @ ket_0_sz
ket_1_sz_prime = unitary @ ket_1_sz
```

```
ket_0_sz_prime, ket_1_sz_prime
```

```
(array([0.70710678, 0.70710678]), array([-0.70710678, 0.70710678]))
```

```
np.linalg.eig?
```

6.4 Dirac notation of Hermitian operator

Write Dirac Notation for the Hermitian operator

$$\tilde{Q} = \begin{pmatrix} \alpha & \gamma \\ \gamma^* & \beta \end{pmatrix},$$

where $\alpha, \beta, \gamma \in \mathcal{C}$.

6.4.1 Solution

The matrix can be decomposed into linear combination of projection operators.

The projection operators are

$$\begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}, \quad \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \quad \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}, \quad \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix},$$

Thus, the operator $\tilde{Q}$ is given by

$$\tilde{Q} = \alpha \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} + \gamma \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} + \gamma^* \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix} + \beta \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}$$

The projection operators can be written in the Dirac notation as

$$\begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} = |0\rangle\langle 0|$$

since

$$0 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad 1 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

Similarly

$$\begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} = 01, \quad \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix} = 10, \quad \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} = 11.$$

Therefore, the operator $\tilde{Q}$ is given by

$$\tilde{Q} = \alpha 00 + \gamma 01 + \gamma^* 10 + \beta 11$$

6.5 Hamiltonian representations in matrix and Dirac notation

Reference: Griffiths - Example 3.8, Problem 3.23

6.5.1 Matrix representation

Imagine a system in which there are two linearly independent states given by

$$0 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad 1 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

The most general state is a normalized linear combination

$$\mathcal{S} = \alpha 0 + \beta 1 = \begin{pmatrix} a \\ b \end{pmatrix}$$

with

$$|\alpha|^2 + |\beta|^2 = 1$$

If the Hamiltonian of the system is given by

$$\tilde{H} = \begin{pmatrix} h & g \\ g & h \end{pmatrix},$$

where g, h are real constants. Find the eigenvalues and eigenkets of Hamiltonian.

6.5.2 Dirac notation

The Hamiltonian for a certain two-level system is

$$\tilde{H} = E (00 + 01 + 10 - 11),$$

where $0, 1$ is an orthonormal basis and E is a number with the dimensions of energy. Find its eigenvalues and eigenvectors. What is the matrix representing $\tilde{H}$ with respect to this basis?

6.6 Differentiation and integration of functions

Differentiate x^4 , $x^3 \sin x$, and e^{-x^2} .

Find $\int x^4 dx$, $\int \sin^3 x dx$, and $\int e^{-x^2} dx$.

6.7 Traveling waves vs standing waves

Reference: Griffiths - Problem 2.18

Show that $[Ae^{ikx} + Be^{-ikx}]$ and $[C \cos kx + D \sin kx]$ are equivalent ways of writing the same function of x , and determine the constants C and D in terms of A and B , and vice versa.

Comment: In quantum mechanics, when $V = 0$, the exponentials represent **traveling waves**, and are most convenient in discussing the free particle, whereas sines and cosines correspond to **standing waves**, which arise naturally in the case of infinite potential well.

6.8 Anti Hermitian operators

Reference: Griffiths - Problem 3.26

An anti-Hermitian (or skew-Hermitian) operator is equal to minus its hermitian conjugate i.e.

$$\tilde{Q}^\dagger = -\tilde{Q}.$$

- Show that the expectation value of an anti-Hermitian operator is imaginary.
- Show that the commutator of two hermitian operators is anti-Hermitian. How about the commutator of two anti-Hermitian operators?

ASSIGNMENTS

Quantum principles

- *Nature of light-matter interaction*
- *Physics of smartphone*
- *Formulation of linear systems*
- *Postulates of quantum mechanics*

Quantum structures

- *Gaussian wavepacket in free space*
- *TISE of quantum structures*

Quantum computing

- *Simulation of one qubit circuits*
- *Simulation of multiple qubit circuits*

7.1 Assignment on light-matter interaction

In this assignment, let us focus on the nature of light and its interaction with matter.

[Download assignment on light-matter interaction](#)

7.2 Assignment on physics of smartphone

In this assignment, let us focus on quantum mechanics principles used in various technologies of smartphone.

[Download assignment on physics of smartphone](#)

7.2.1 Solutions

- [Download solution to assignment on physics of smartphone](#)
- [Download detailed solution to assignment on physics of smartphone](#)

7.3 Assignment on linear system formulation

In this notebook, let us look at how to “linearize” systems from various disciplines.

[Download assignment on linear system formulation](#)

7.3.1 Solutions

- [Download solution to assignment on linear system formulation](#)

7.4 Assignment on postulates of quantum mechanics

In this notebook, let us explore the postulates of quantum mechanics.

[Download assignment on postulates of quantum mechanics](#)

7.4.1 Solutions

- [Download solution to assignment on postulates of quantum mechanics](#)

7.5 Assignment on Gaussian wavepacket in free space

In this notebook, let us study the time evolution of Gaussian wavepacket in free space.

[Download assignment on Gaussian wavepacket](#)

7.6 Assignment on TISE of quantum structures

In this assignment, we solve for Time Independent Schrodinger equation (TISE) for the following quantum structures

1. step potential
2. infinite potential well
3. finite potential well
4. finite potential barrier
5. harmonic potential

7.6.1 Finite difference method

Here, we have used the **Numerov algorithm** to obtain the recurrence relation between adjacent wave function values.

7.6.2 Atomic units

Also the fundamental constants are measured in atomic units. This allows to choose the step size for finite difference independent of the scale of the problem.

In the atomic units, we consider the electron mass, reduced Planck's constant and Bohr's radius each as 1 unit.

$$m_e = 1, \quad \hbar = 1, \quad a_0 = 1$$

7.6.3 Solvers

There are two variants of TISE solvers

- Python version
- Html + js version

The first version requires environment with numpy and matplotlib, whereas the second version can run on browser (Thanks to JoyBoy).

7.6.4 Step potential

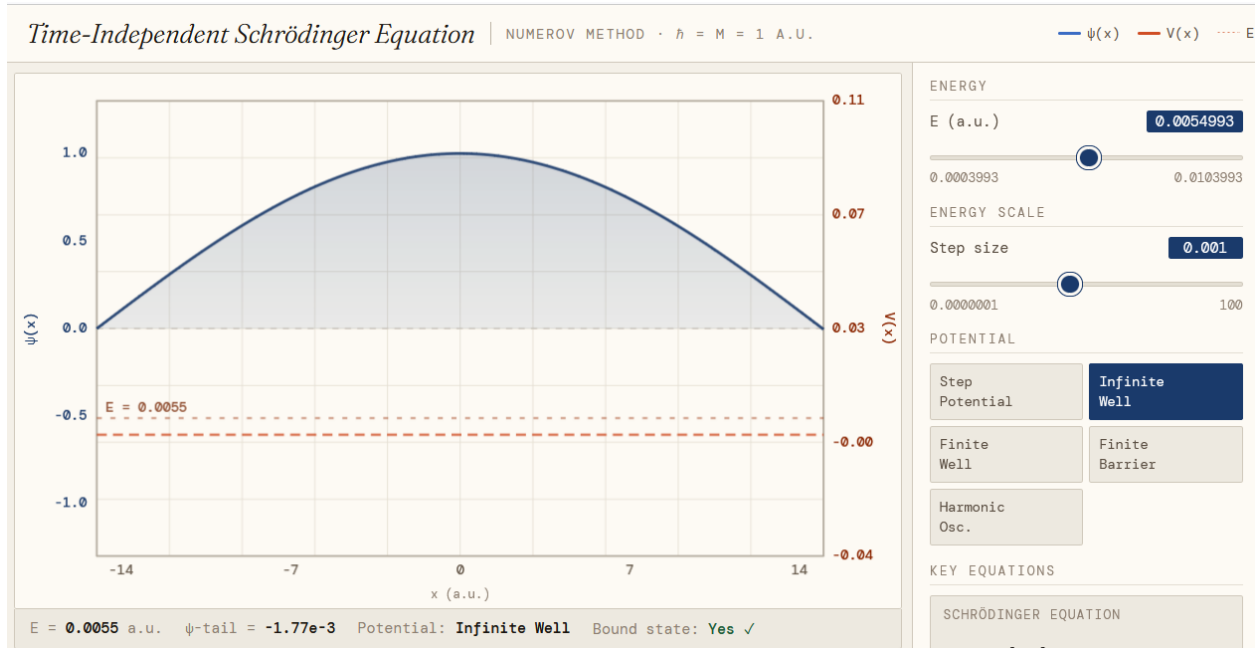
In the lecture on *scattering states*, we have discussed about quantum penetration and quantum scattering.

Assignment

Demonstrate quantum penetration and quantum scattering for the step potential.

7.6.5 Infinite potential well

In the assignment, numerically find the eigenvalues and eigen functions of the TISE using the “wag-the-dog” or “shooting star” method



The infinite potential function is given by

$$V(x) = \begin{cases} 0 & 0 < x < a \\ \infty & \text{otherwise} \end{cases}$$

In the given case,

$$a = 30$$

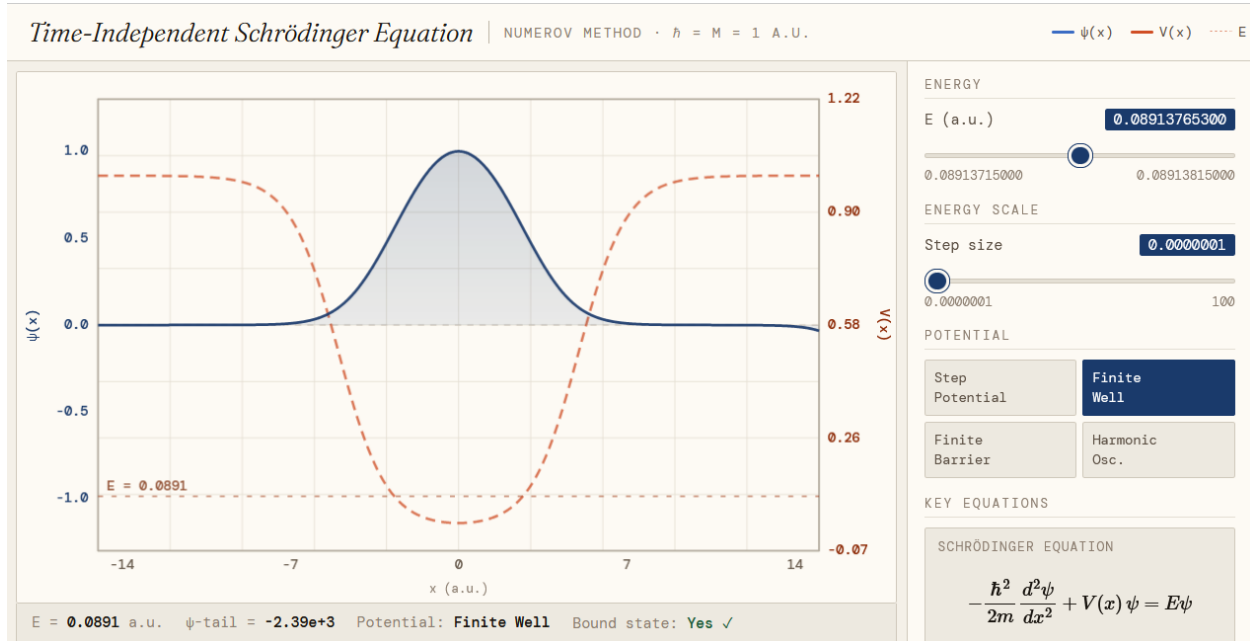
Hence, the energy eigen values are given by

$$E_n = \frac{n^2 \pi^2}{2 \cdot 900}$$

Assignment

Find the first five energy eigenvalues and eigenfunctions **numerically** and check if they match the analytical solution.

7.6.6 Finite potential well



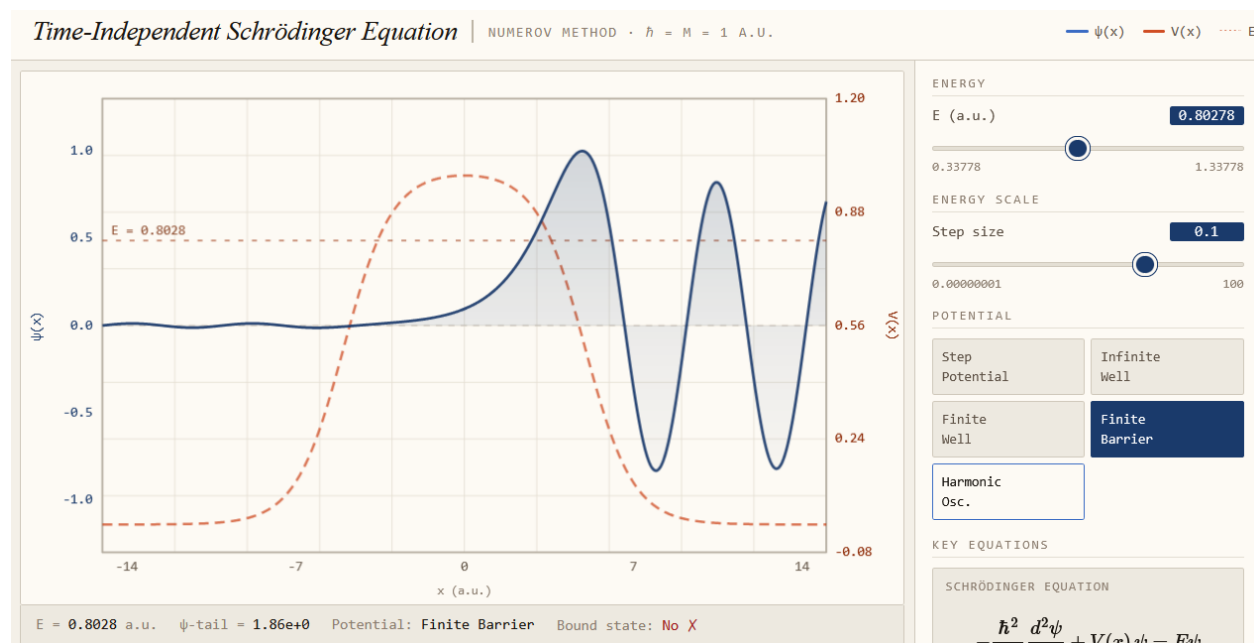
Assignment

Find the numerical solution for the first five energy eigenvalues and eigenfunctions of finite potential well, and compare with the solution of infinite potential well.

7.6.7 Finite potential barrier

Assignment

Demonstrate quantum mechanical tunneling for the finite potential barrier even in the case of $E < V_0$.

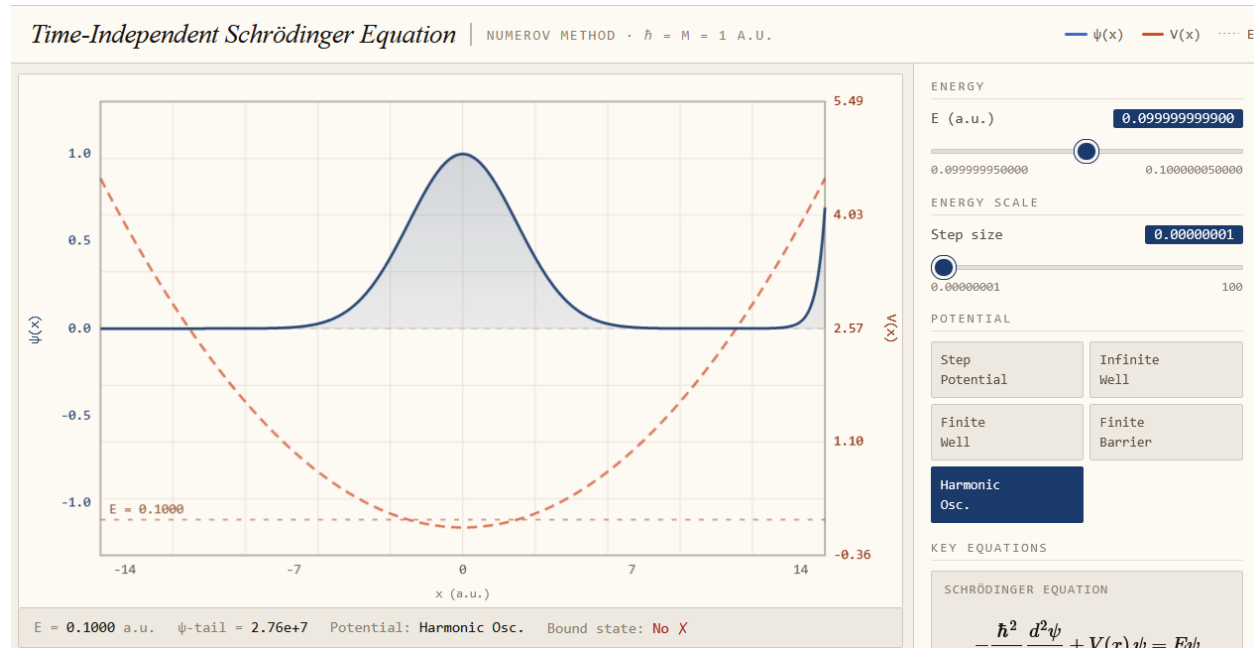


7.6.8 Harmonic potential

Show that the energy eigenvalues of harmonic potential have equal difference between them i.e.

$$E_n - E_{n-1} = \text{constant}$$

where E_n is the nth eigen energy.



Assignment

- Find the numerical solution for the first five energy eigenvalues and eigenfunctions of harmonic potential.
- Confirm Equation A.
- Find the constant.
- If the constant is shown to be $\hbar\omega$, where ω is the classical version of the angular frequency, then find ω .
- Confirm that the ground state energy is equal to $\frac{1}{2}\hbar\omega$
- We have chosen the harmonic potential as

$$V(x) = \frac{1}{2}kx^2,$$

where k is the spring constant.

If $\omega = \sqrt{\frac{k}{m}}$, then find k

7.7 Assignment on single qubit circuits

Quantum computation is based on transformation of wavekets by quantum gates.

In this assignment, let us focus on single qubit circuit that consist of single quantum bit and single quantum gate.

#TODO Needs to be designed.

7.8 Assignment on multiple qubit circuits

In this assignment, let us focus on multiple qubit circuits.

#TODO Needs to be designed.